

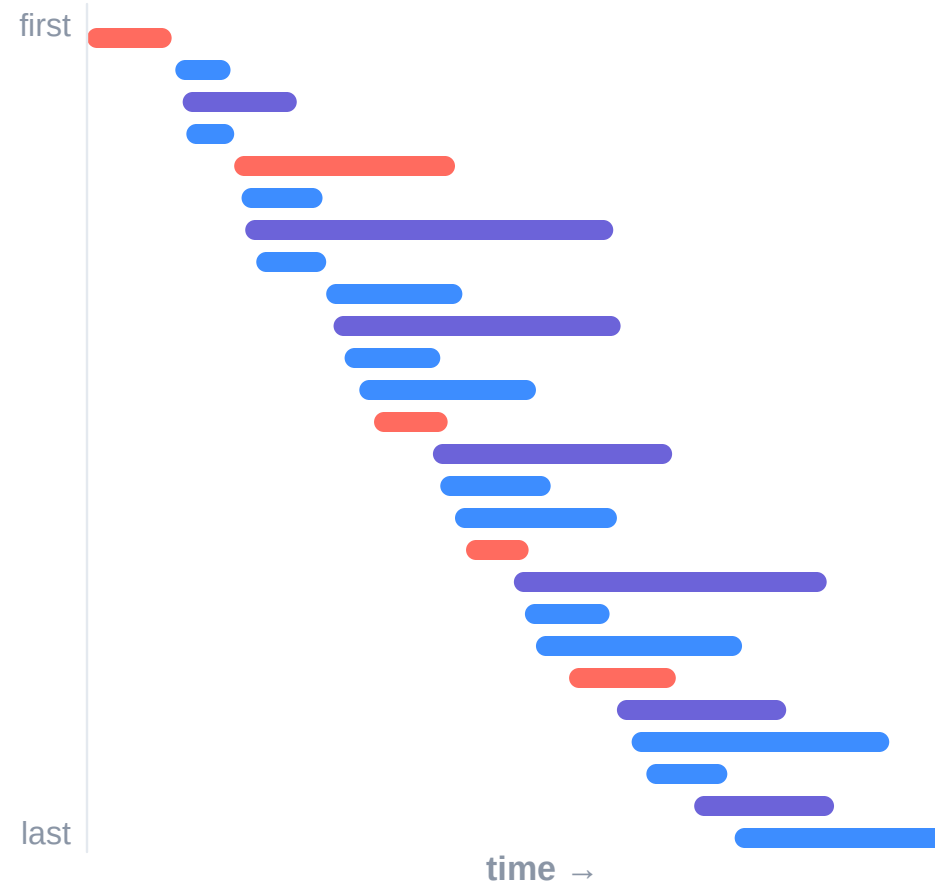
The Web and HTTP

Objects, requests, and the protocol that moves them

The web is where almost every application now runs, and HTTP is the protocol that carries it. This lecture covers why applications moved into the browser, what a web page is made of, and how HTTP fetches each of its files.

Yaxiong Xie

ONE PAGE LOAD
26 of the 72 requests it makes



Why does the web get its own lecture?

01

Why the web

The web is where almost every application people use now runs. This part is about why that happened, and what the browser had to become for it to happen.

There are 1.49 billion websites

1,494,915,628

websites answering an HTTP request, July 2026

305,348,459

unique domains

14,772,048

web-facing computers

+5,500,000

sites added in one month

A site is a hostname that answers an HTTP request.

Every one of these is reached with the same protocol.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

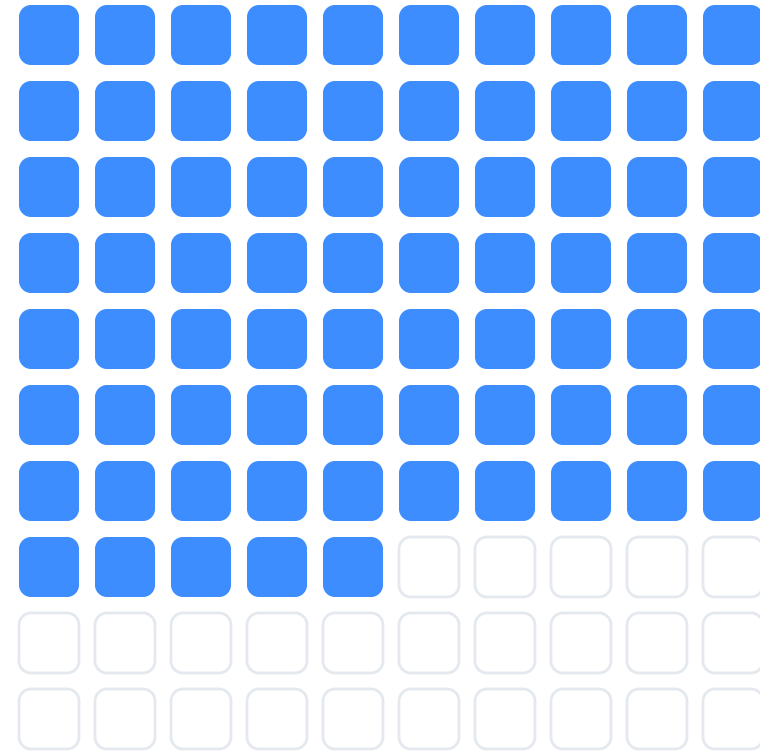
About three-quarters of the world is online

75%

of the world population is online

2.2 billion people are still offline.

Nearly all of them reach it through a browser
or an application that speaks the same protocol.



ONE SQUARE = 1% OF THE WORLD POPULATION

The web is the interface almost all of them use.

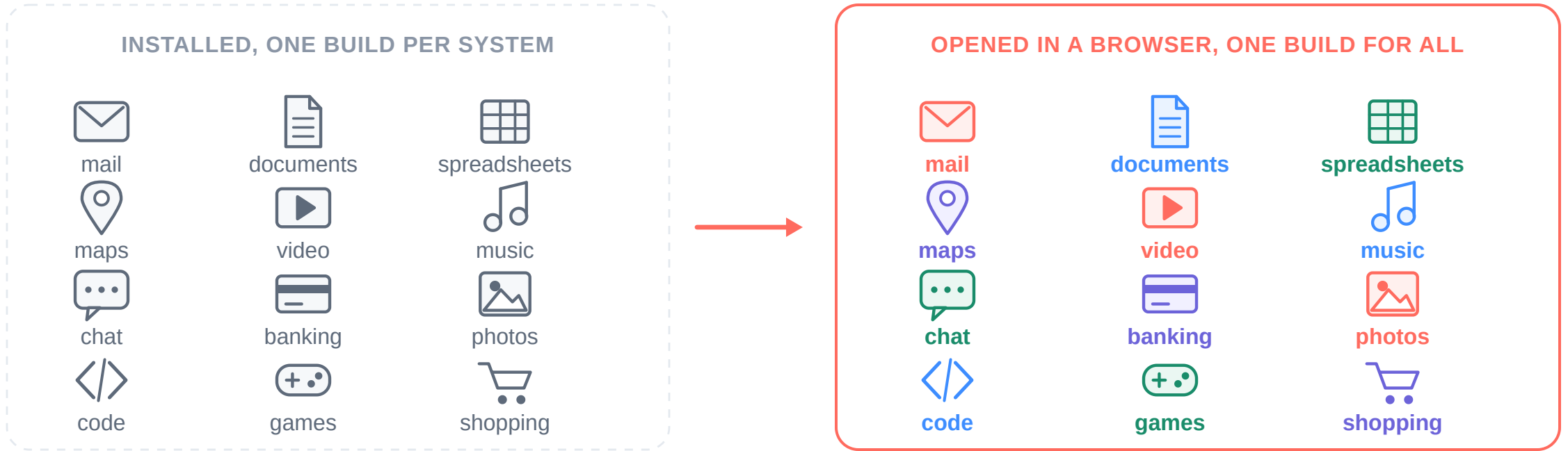
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser runs applications that used to be installed



The same twelve applications, with nothing downloaded or installed.

The browser replaced the installer for most categories of software.

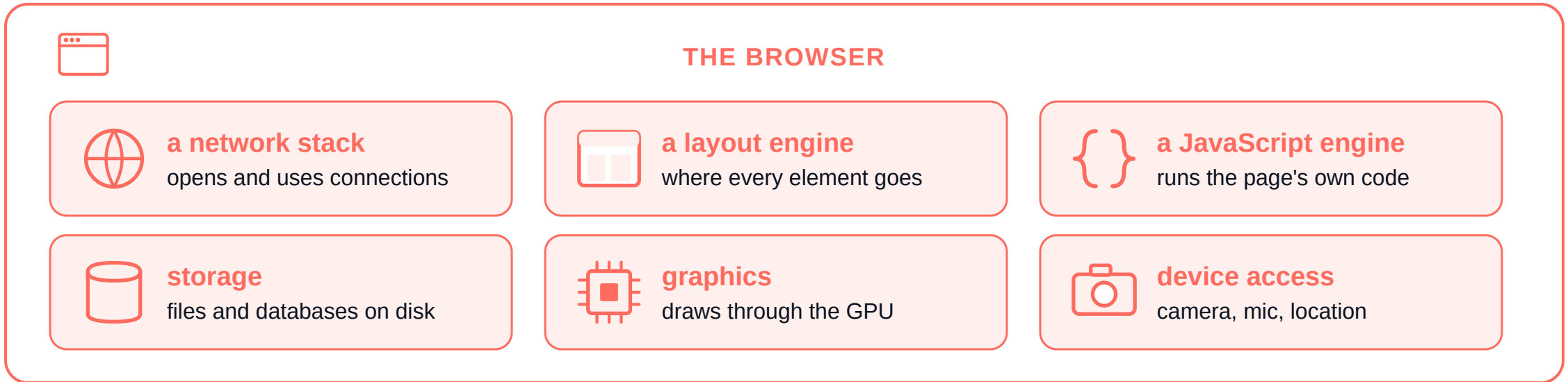
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser is a runtime, not a document viewer



An installed program gets these six things from the operating system. A page gets them from the browser.

Anything the browser can do, a page can do.

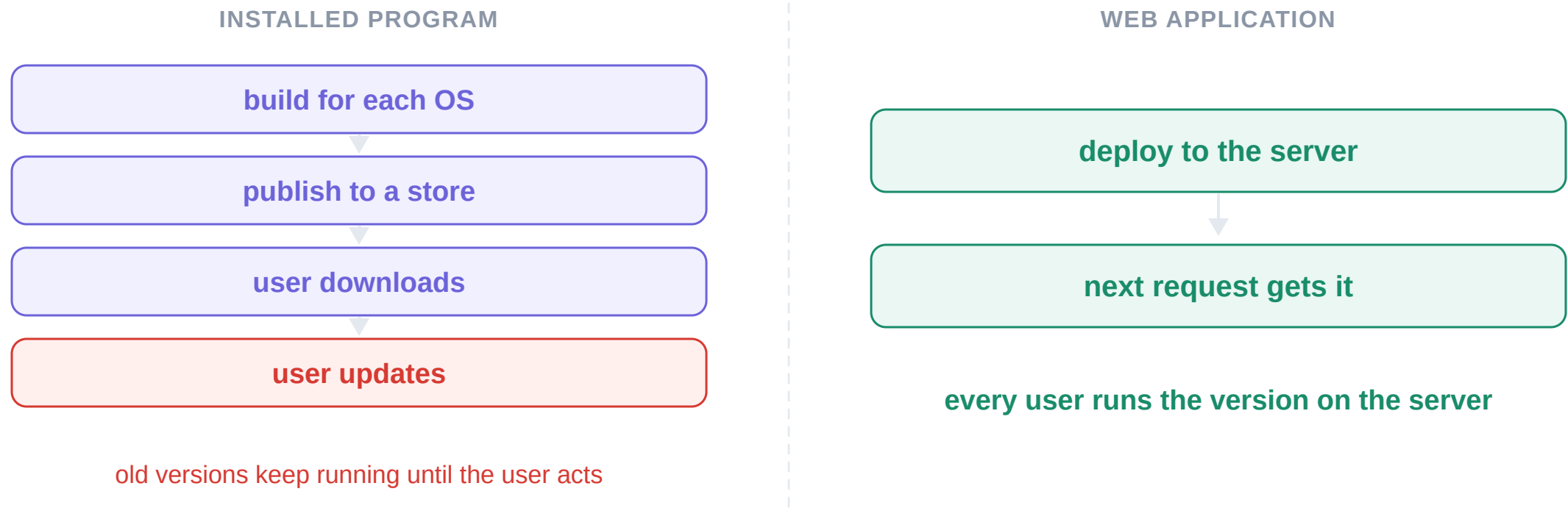
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The program is fetched on every use, not installed once



Deployment is a file change on one machine.

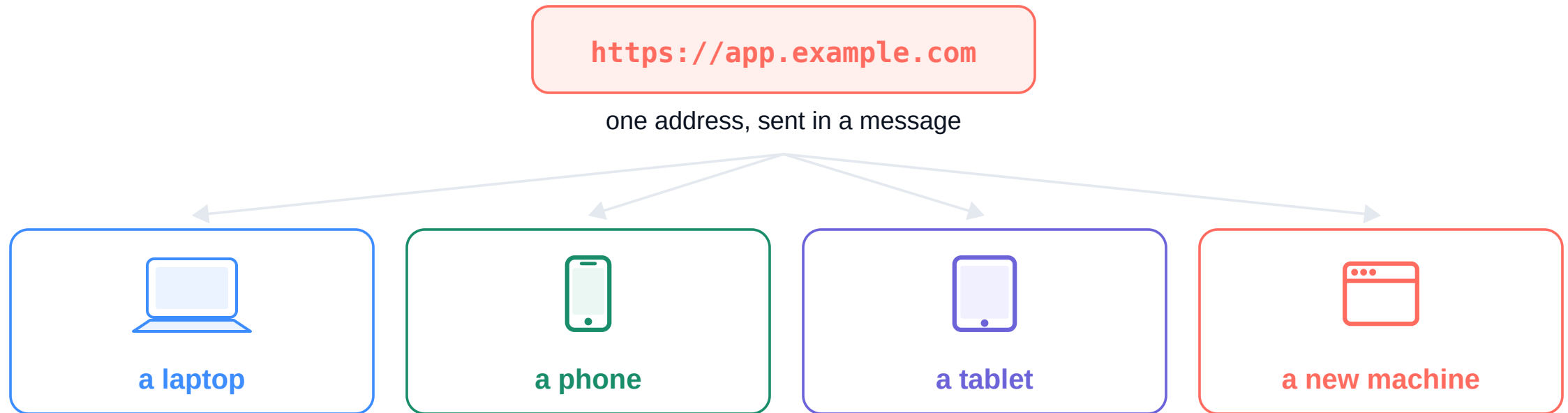
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Distributing a web application is sending a link



no store has to approve it · no version has to be chosen · nothing is installed first

The address is the whole distribution mechanism.

One address reaches every device that has a browser.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

What the rest of this lecture answers

02

What is a web page, and how is it built?

HTML, objects, and how many of them

03

How do those files reach the browser?

HTTP requests, responses and status codes

04

What does HTTP do beyond moving bytes?

connections, caching, cookies

Part 2 starts with the page itself, before any protocol.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

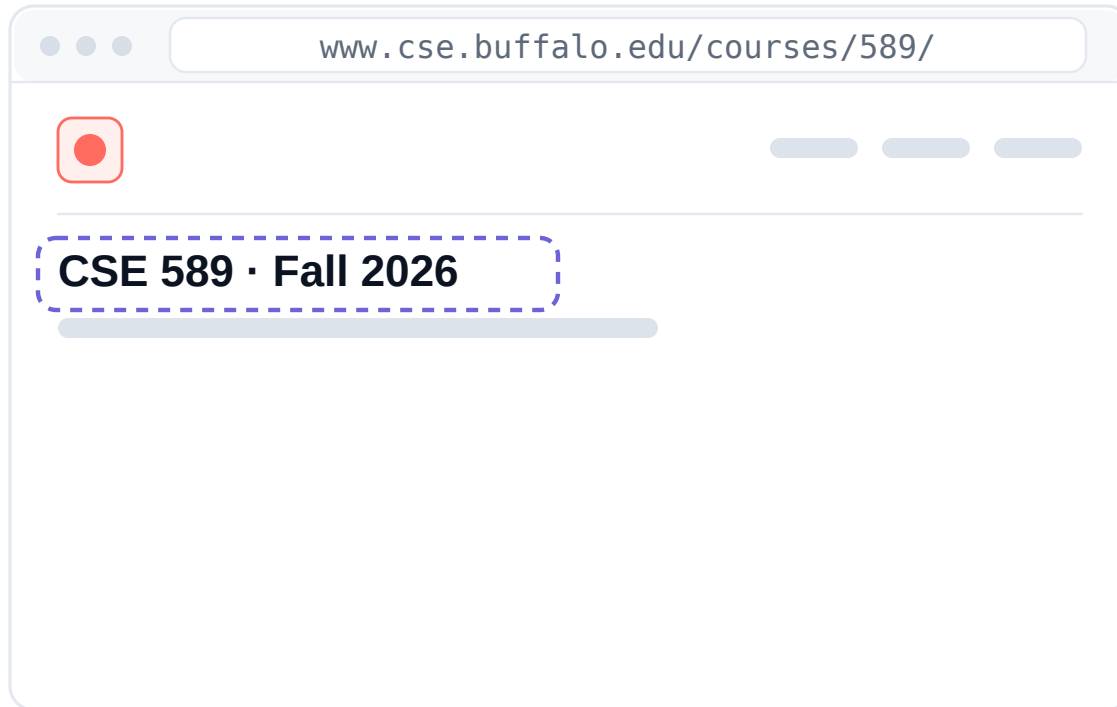
What is a web page, and how is it built?

02

How a page works

A page is a text file that the browser turns into what you see, plus a set of separate files it names. This part is the page itself, with no protocol in it yet.

The browser draws one thing for each tag it reads



```
<!doctype html>
<html><head>
<title>CSE 589</title>
</head><body>
<h1>CSE 589 · Fall 2026</h1>

<p>Networks, from the top down.</p>
<a href="/hw1.html">Homework 1</a>
</body></html>
```

<h1>

A tag marks a piece of content.
This one marks a heading: large, bold, on a line of its own.

● ○ ○ ○ step 1 of 4

The file says what each piece is; the browser decides how it looks.

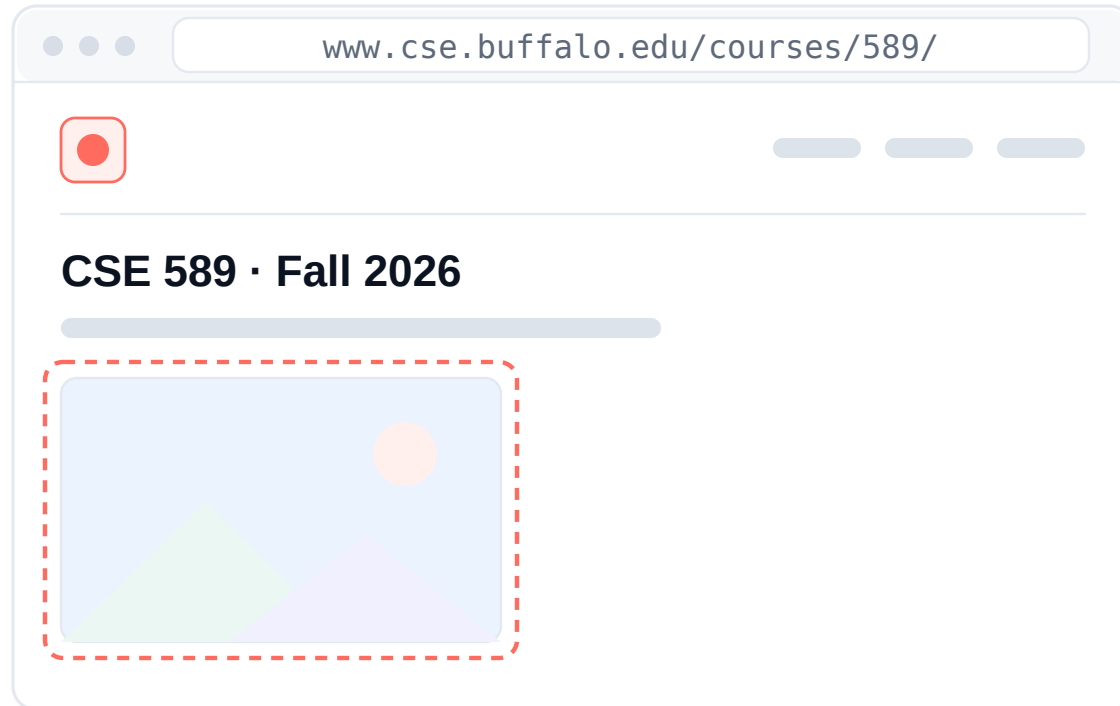
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser draws one thing for each tag it reads



```
<!doctype html>
<html><head>
<title>CSE 589</title>
</head><body>
<h1>CSE 589 · Fall 2026</h1>

<p>Networks, from the top down.</p>
<a href="/hw1.html">Homework 1</a>
</body></html>
```

This tag does not contain a picture. It names a file.
The browser fetches that file and draws it where the tag stood.

The file says what each piece is; the browser decides how it looks.

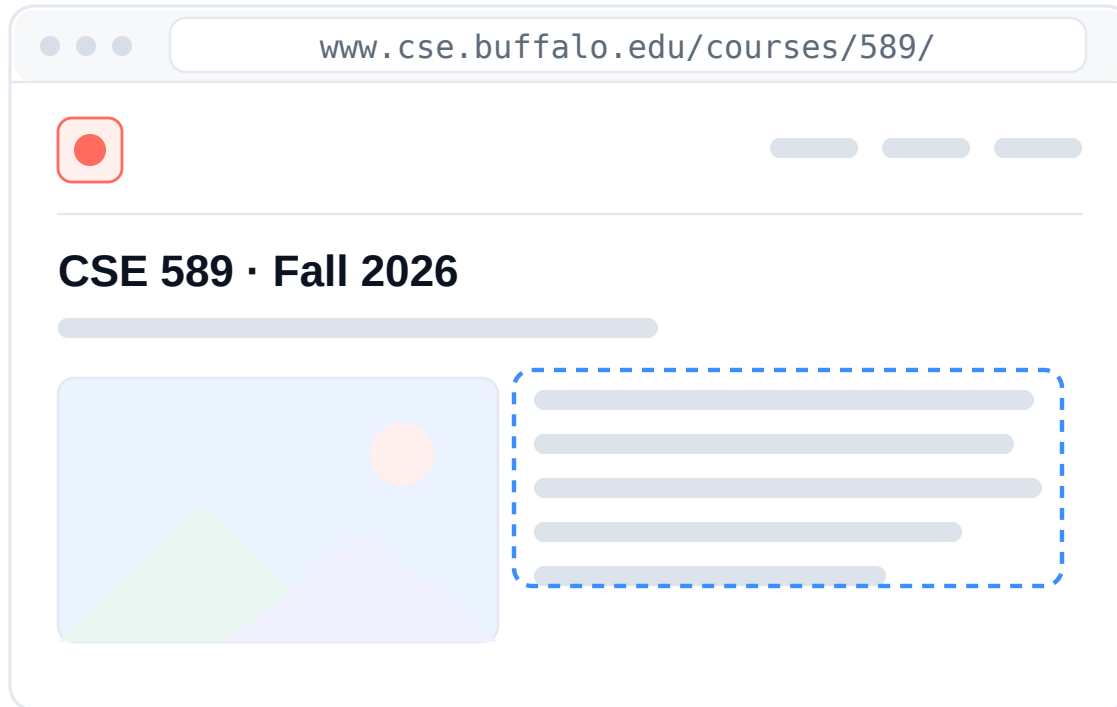
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser draws one thing for each tag it reads



step 3 of 4

```
<!doctype html>
<html><head>
<title>CSE 589</title>
</head><body>
<h1>CSE 589 · Fall 2026</h1>

<p>Networks, from the top down.</p>
<a href="/hw1.html">Homework 1</a>
</body></html>
```

<p>

This tag marks a paragraph.
The browser breaks the text into lines that fit the width it has.

The file says what each piece is; the browser decides how it looks.

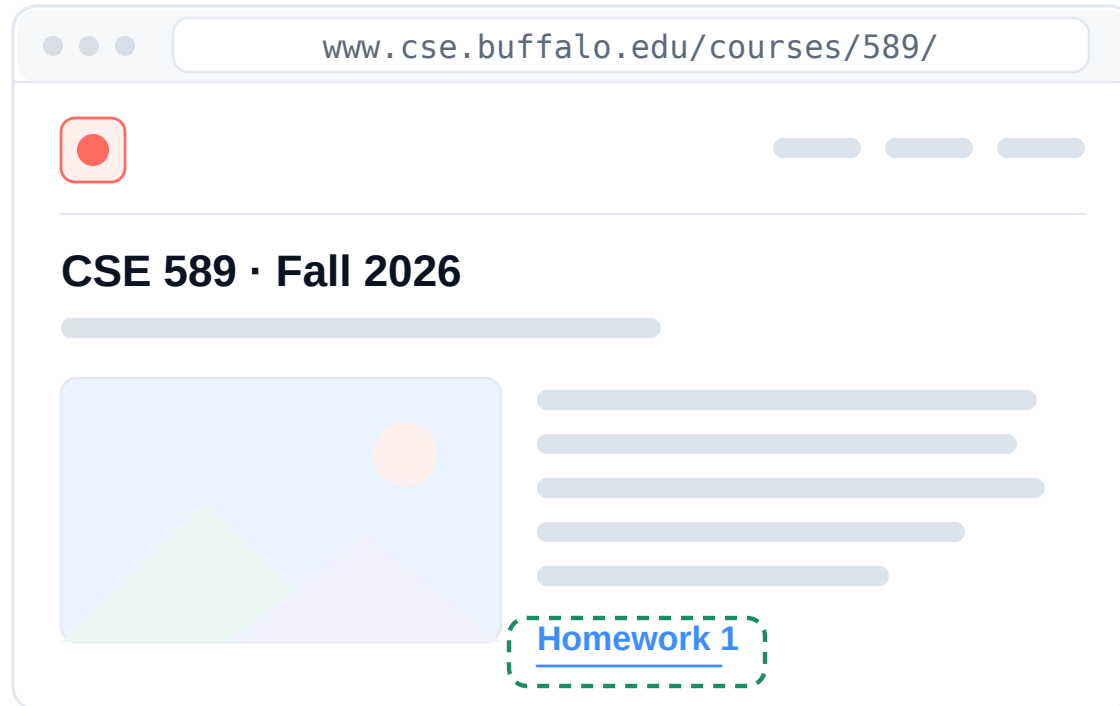
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser draws one thing for each tag it reads



```
<!doctype html>
<html><head>
<title>CSE 589</title>
</head><body>
<h1>CSE 589 · Fall 2026</h1>

<p>Networks, from the top down.</p>
<a href="/hw1.html">Homework 1</a>
</body></html>
```

This tag marks a link.

Clicking it makes the browser load the URL in the href.

step 4 of 4

The file says what each piece is; the browser decides how it looks.

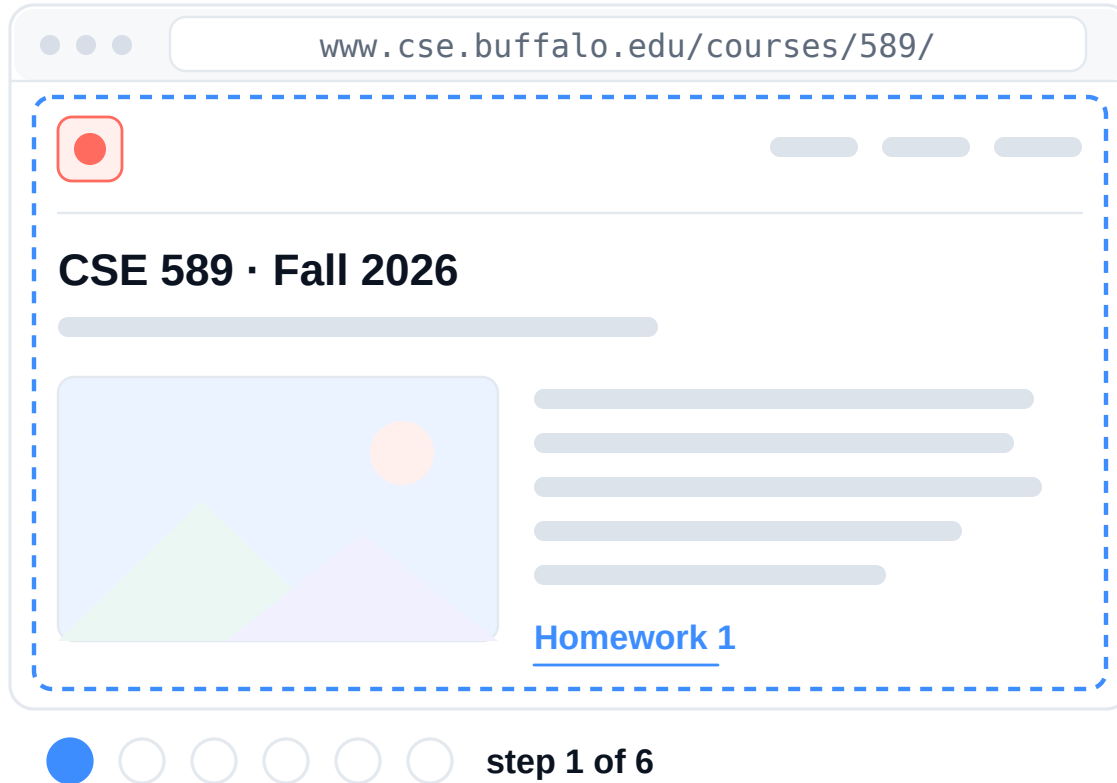
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The page on screen is six separate files



● index.html	the text and every reference below	22 KB
● photo.jpg	the picture	310 KB
● logo.png	the mark at the top left	14 KB
● font.woff2	the shapes of the letters	28 KB
● style.css	the colours, spacing and layout	77 KB
● app.js	the menu that opens when clicked	210 KB

1 of 6 · 22 KB so far

the file the browser asked for; it names the other five

The browser asked for one file. It is not the page.

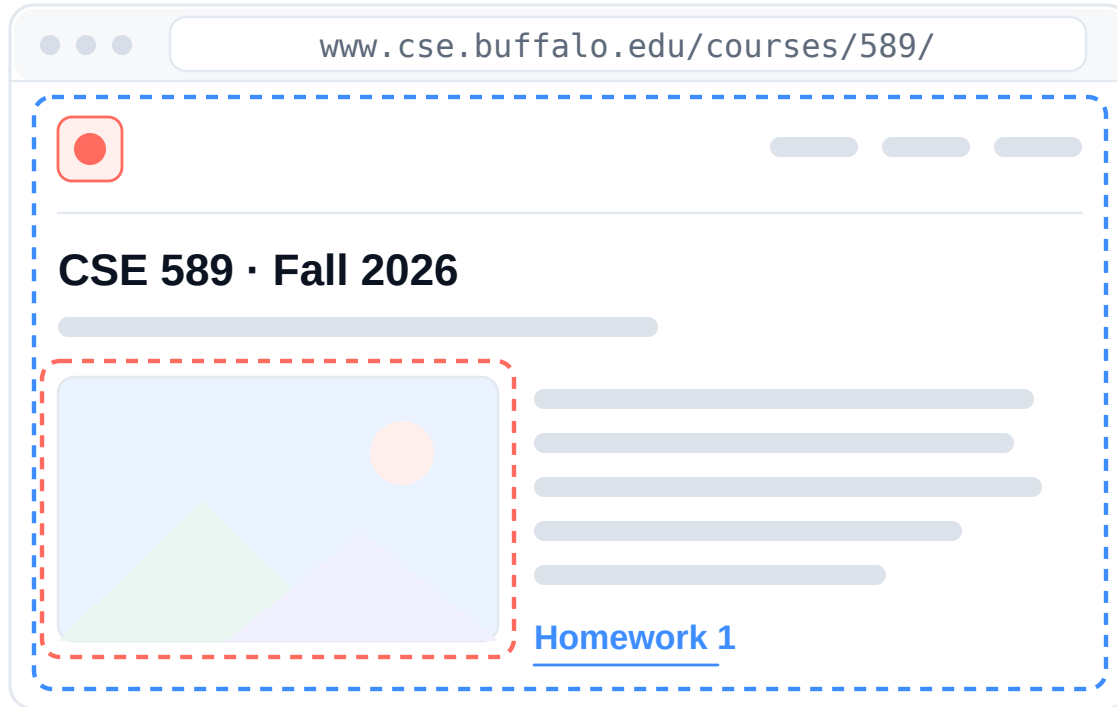
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The page on screen is six separate files



● ● ○ ○ ○ ○ step 2 of 6

● <code>index.html</code>	the text and every reference below	22 KB
● <code>photo.jpg</code>	the picture	310 KB
● <code>logo.png</code>	the mark at the top left	14 KB
● <code>font.woff2</code>	the shapes of the letters	28 KB
● <code>style.css</code>	the colours, spacing and layout	77 KB
● <code>app.js</code>	the menu that opens when clicked	210 KB

2 of 6 · 332 KB so far

a second request, drawn where the `<img>` tag stood

The picture is a second file, fetched on its own.

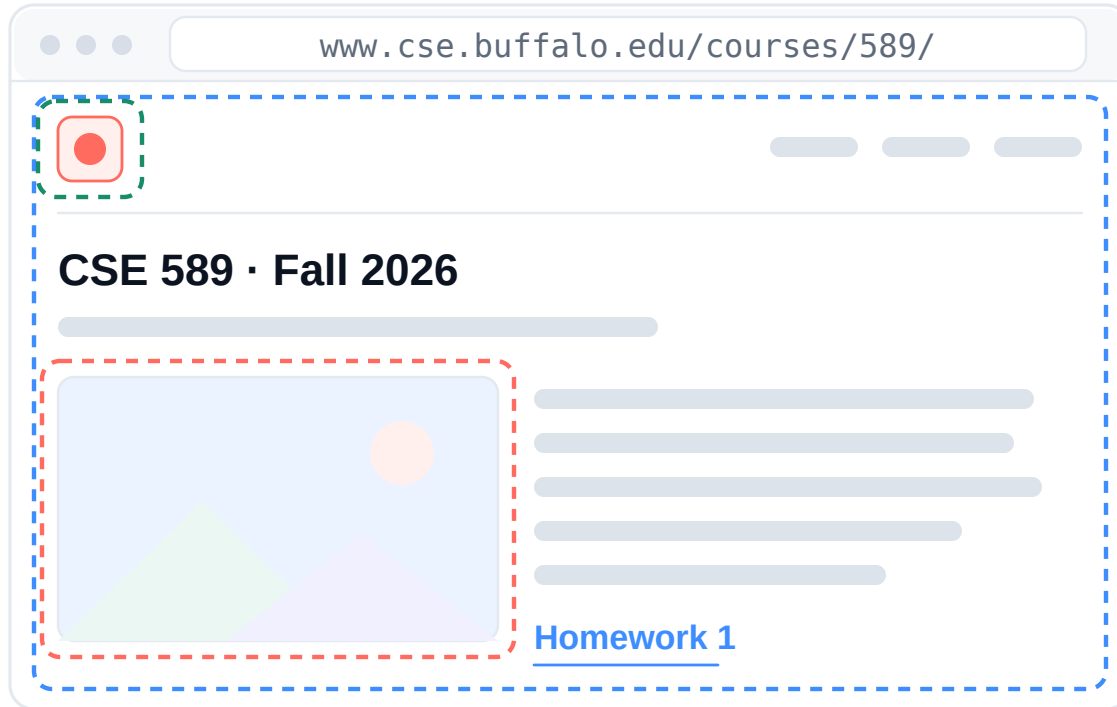
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The page on screen is six separate files



● <code>index.html</code>	the text and every reference below	22 KB
● <code>photo.jpg</code>	the picture	310 KB
● <code>logo.png</code>	the mark at the top left	14 KB
● <code>font.woff2</code>	the shapes of the letters	28 KB
● <code>style.css</code>	the colours, spacing and layout	77 KB
● <code>app.js</code>	the menu that opens when clicked	210 KB

3 of 6 · 346 KB so far

another request, for the mark at the top left

Three files so far, and nothing has been styled yet.

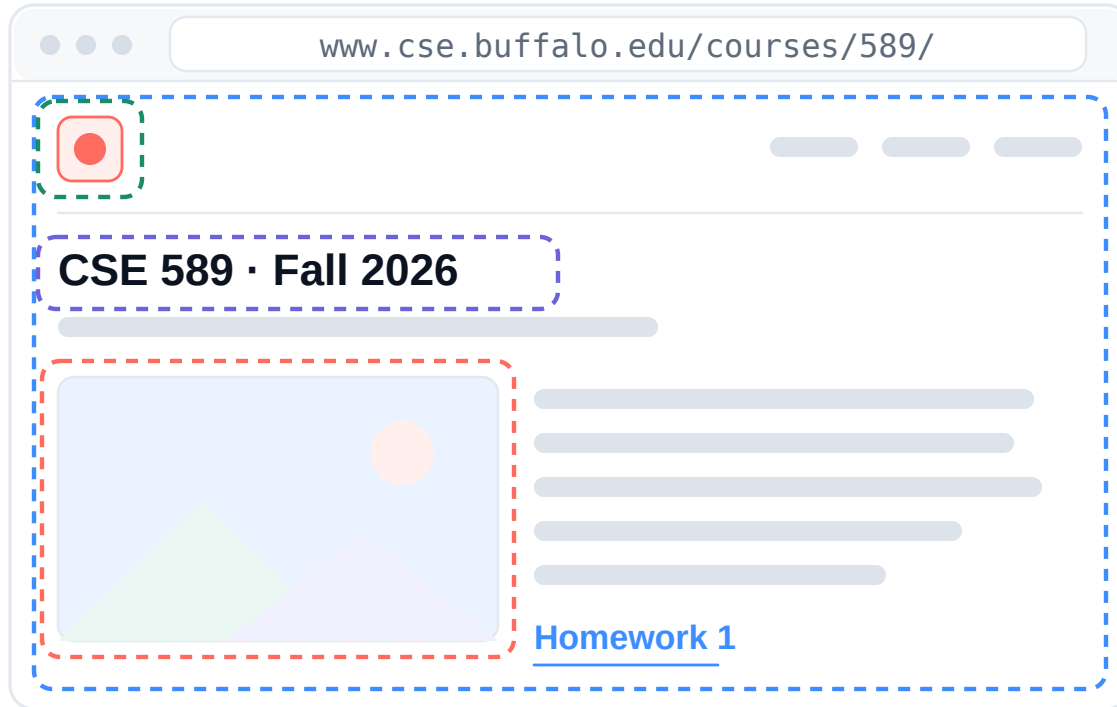
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The page on screen is six separate files



● ● ● ● ○ ○ step 4 of 6

● <code>index.html</code>	the text and every reference below	22 KB
● <code>photo.jpg</code>	the picture	310 KB
● <code>logo.png</code>	the mark at the top left	14 KB
● <code>font.woff2</code>	the shapes of the letters	28 KB
● <code>style.css</code>	the colours, spacing and layout	77 KB
● <code>app.js</code>	the menu that opens when clicked	210 KB

4 of 6 · 374 KB so far

the letter shapes; without it the text falls back

Even the shapes of the letters are a separate download.

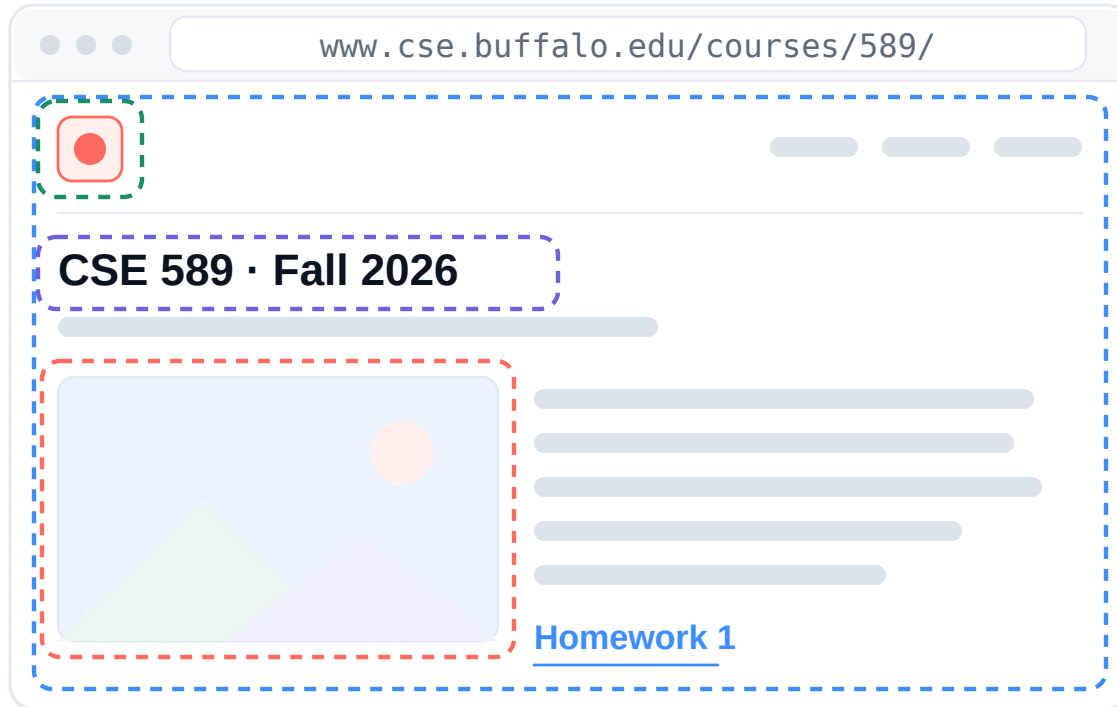
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The page on screen is six separate files



● ● ● ● ● ● step 5 of 6

● <code>index.html</code>	the text and every reference below	22 KB
● <code>photo.jpg</code>	the picture	310 KB
● <code>logo.png</code>	the mark at the top left	14 KB
● <code>font.woff2</code>	the shapes of the letters	28 KB
● <code>style.css</code>	the colours, spacing and layout	77 KB
● <code>app.js</code>	the menu that opens when clicked	210 KB

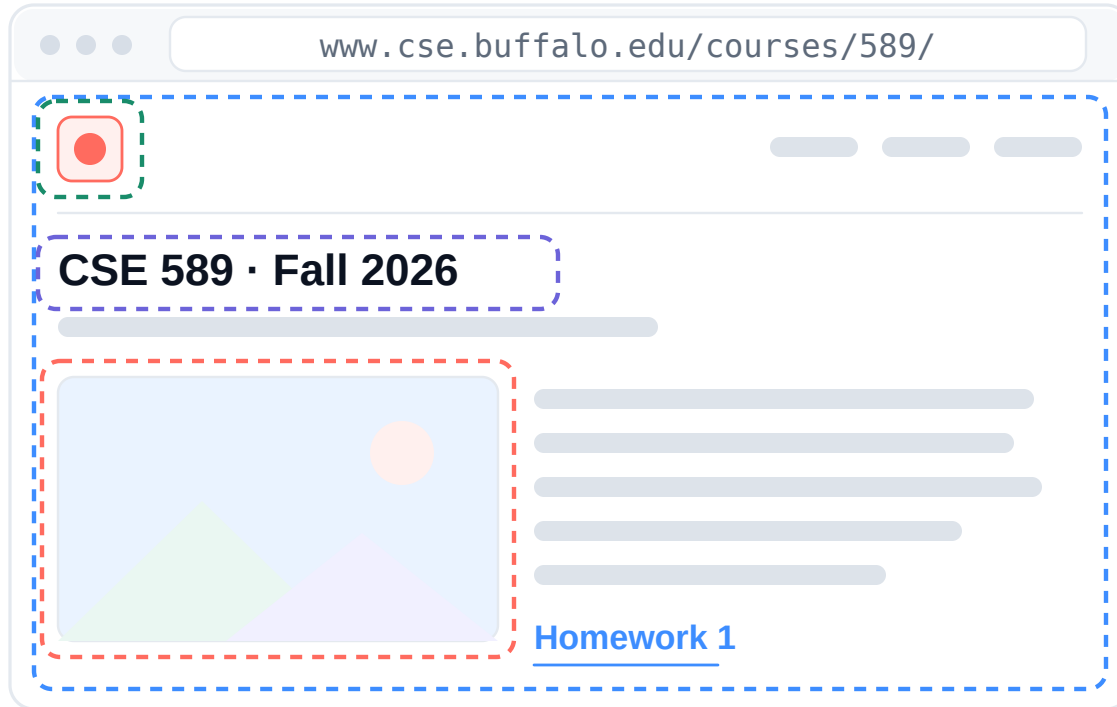
5 of 6 · 451 KB so far

no region of its own: it sets how everything looks

Two of the six have no region of their own.

- 1 Why the web
- 2 How a page works
- 3 HTTP
- 4 What HTTP adds

The page on screen is six separate files



● ● ● ● ● ● step 6 of 6

● <code>index.html</code>	the text and every reference below	22 KB
● <code>photo.jpg</code>	the picture	310 KB
● <code>logo.png</code>	the mark at the top left	14 KB
● <code>font.woff2</code>	the shapes of the letters	28 KB
● <code>style.css</code>	the colours, spacing and layout	77 KB
● <code>app.js</code>	the menu that opens when clicked	210 KB

all six · 661 KB

no region either: it runs when the reader clicks

A page is not one file. It is a set of files, listed by the first one.

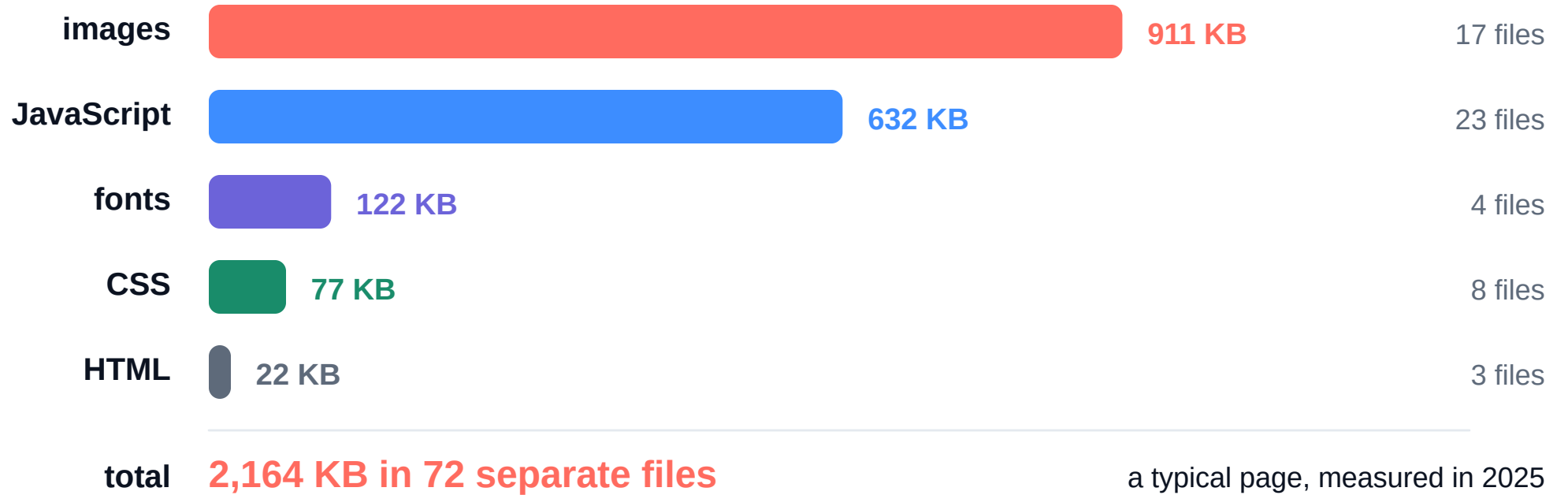
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A typical page needs 72 files, not 6



Half of all pages need more than this, and half need less.

Loading one page means fetching dozens of separate files.

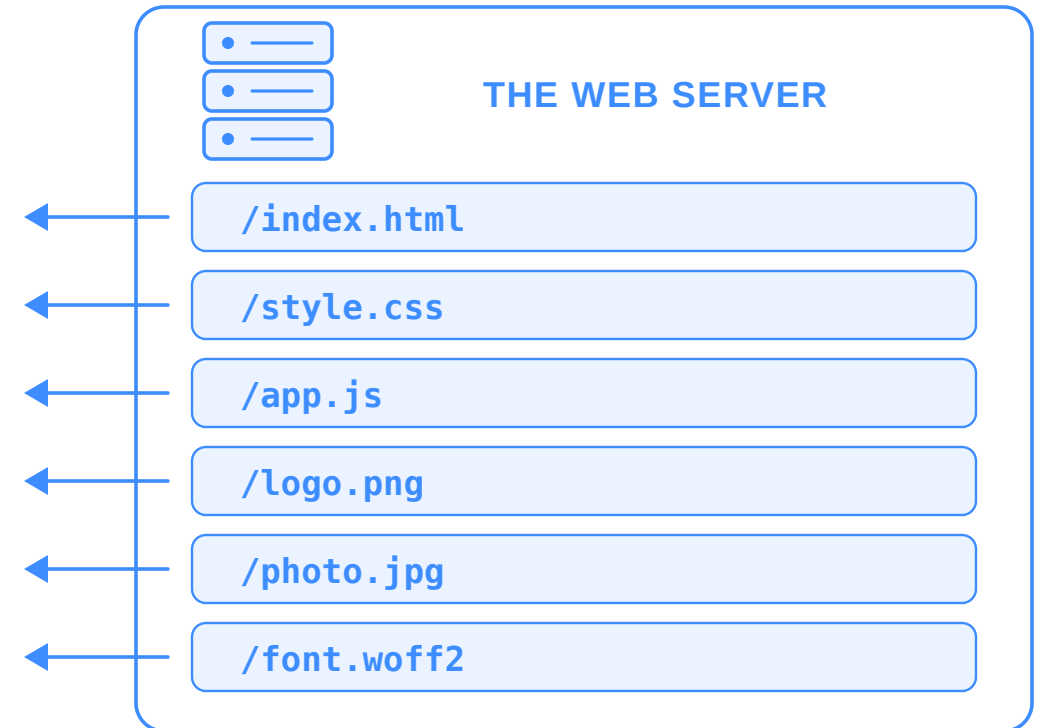
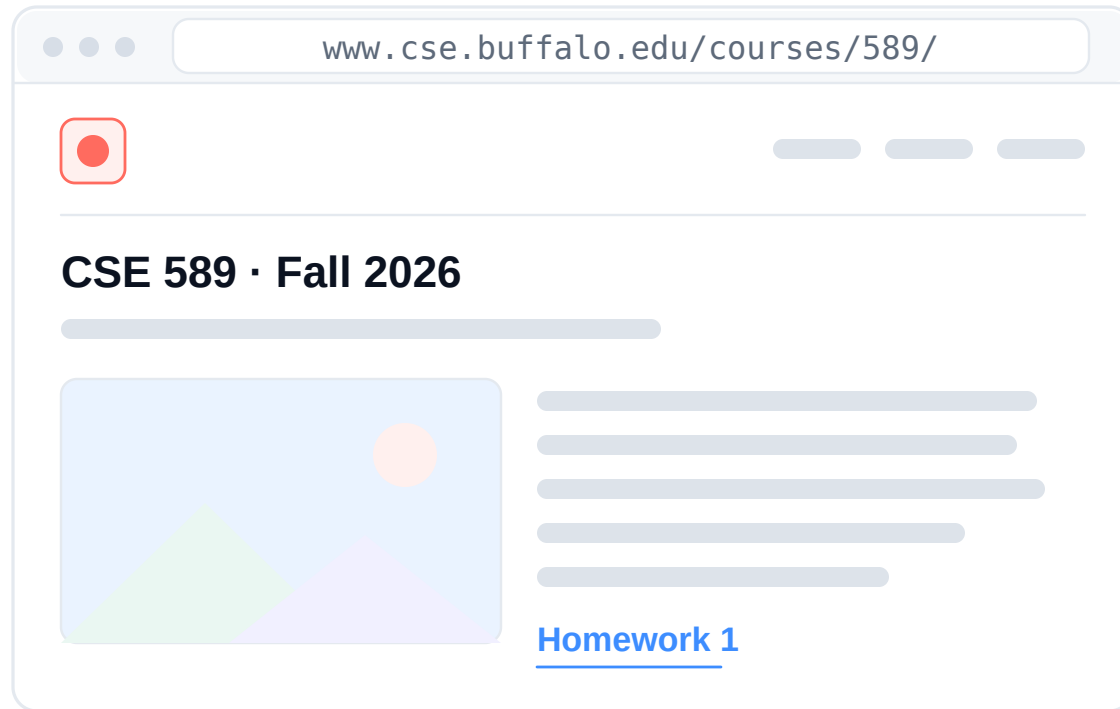
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Every one of those files sits on a web server



The browser starts with none of them and asks for one at a time.

Each object has to be downloaded on its own.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser builds the page in five steps

01

read the text

the file arrives as a stream of characters

02

build the tree

one node for every tag, nested as the tags are

03

fetch the files it names

the pictures, the style sheet, the script

04

compute the layout

a position and a size for every node

05

paint

turn that into pixels



what each step produces



The file arrives as characters. There is nothing on screen yet.

A page starts as a stream of characters.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser builds the page in five steps

01

read the text

the file arrives as a stream of characters

02

build the tree

one node for every tag, nested as the tags are

03

fetch the files it names

the pictures, the style sheet, the script

04

compute the layout

a position and a size for every node

05

paint

turn that into pixels



what each step produces



The tree built here is the DOM. Everything after this works on the tree.

The DOM is the tree, and it is what all the rest of the work happens on.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser builds the page in five steps

01

read the text

the file arrives as a stream of characters

02

build the tree

one node for every tag, nested as the tags are

03

fetch the files it names

the pictures, the style sheet, the script

04

compute the layout

a position and a size for every node

05

paint

turn that into pixels



what each step produces



Only now does the browser know which other files the page needs.

The browser cannot know what to fetch until it has read the HTML.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser builds the page in five steps

01

read the text

the file arrives as a stream of characters

02

build the tree

one node for every tag, nested as the tags are

03

fetch the files it names

the pictures, the style sheet, the script

04

compute the layout

a position and a size for every node

05

paint

turn that into pixels



what each step produces



Layout turns the tree into rectangles: a position and a size for every node.

Layout gives every node a position and a size.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The browser builds the page in five steps

01

read the text

the file arrives as a stream of characters

02

build the tree

one node for every tag, nested as the tags are

03

fetch the files it names

the pictures, the style sheet, the script

04

compute the layout

a position and a size for every node

05

paint

turn that into pixels



what each step produces



Steps 3 and 4 run again as each file arrives, so the page is laid out again.

The page is computed by the browser, not sent as a picture.

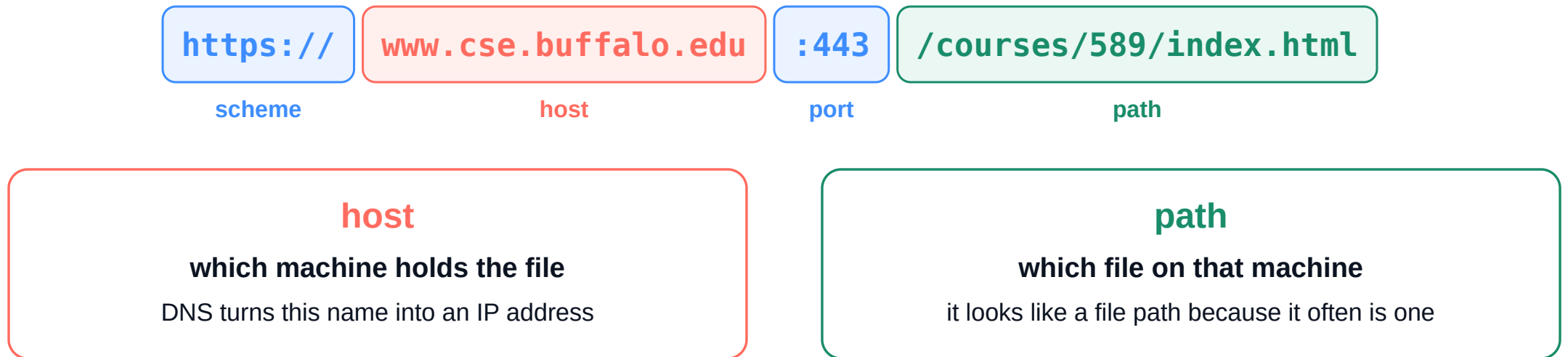
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A URL says which machine, and which file on it



These two are the whole address: a machine, and a file on it.

Every object on a page has a URL of its own.

The port says which program on that machine answers

ONE MACHINE, MANY SERVERS

128.205.32.52

:80

the web server

:25

the mail server

:22

the ssh server

The port says which of them the connection is for.

WHY YOU NEVER TYPE IT

<https://www.cse.buffalo.edu>

the scheme already implies the port

http:// → :80

https:// → :443

ssh:// → :22

A different port has to be written out, as :8080.

A URL names a machine, a program on it, and a file.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The last piece is a way to ask for one file

the page

HTML says what the content is



the objects

each one a separate file



the names

a URL for every one of them



missing

a way to ask for one



THE RULES BOTH SIDES HAVE TO AGREE ON

how a client asks for the object named by one URL

how the server says whether it worked, and what it is sending back

how both sides know where one message ends and the next begins

how to do it dozens of times without starting over each time

That agreed set of rules is HTTP, and Part 3 is about it.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

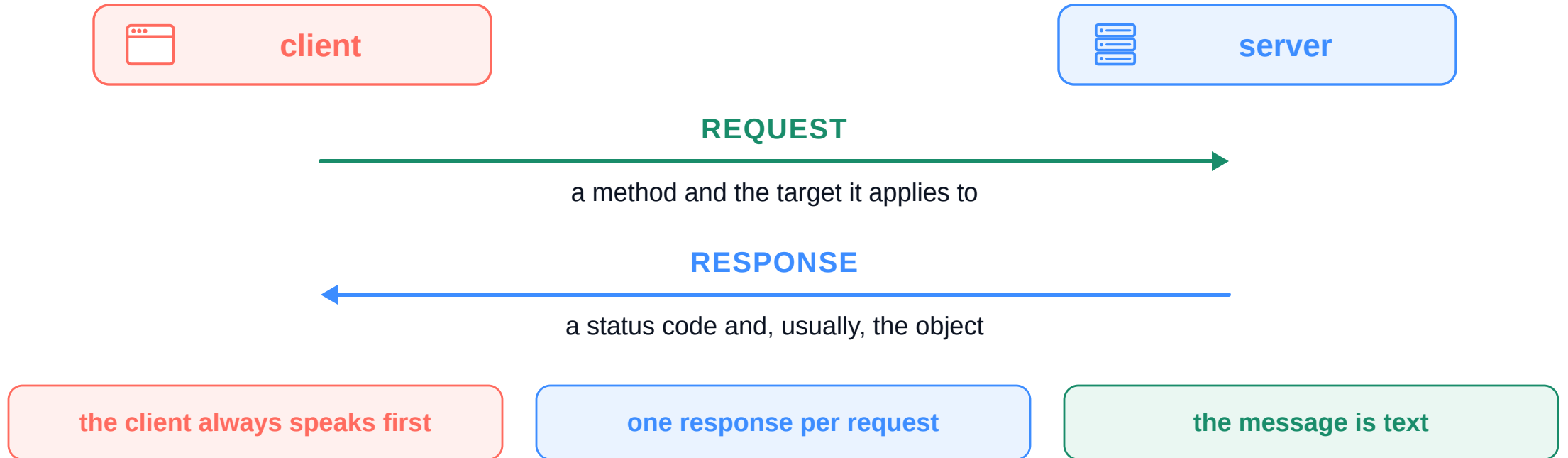
How do those files reach the browser?

03

HTTP

One request, one response, both in text. The request names a method and a target; the response carries a status code and the object itself.

HTTP is one request and one response



The client asks for one object; the server answers once.

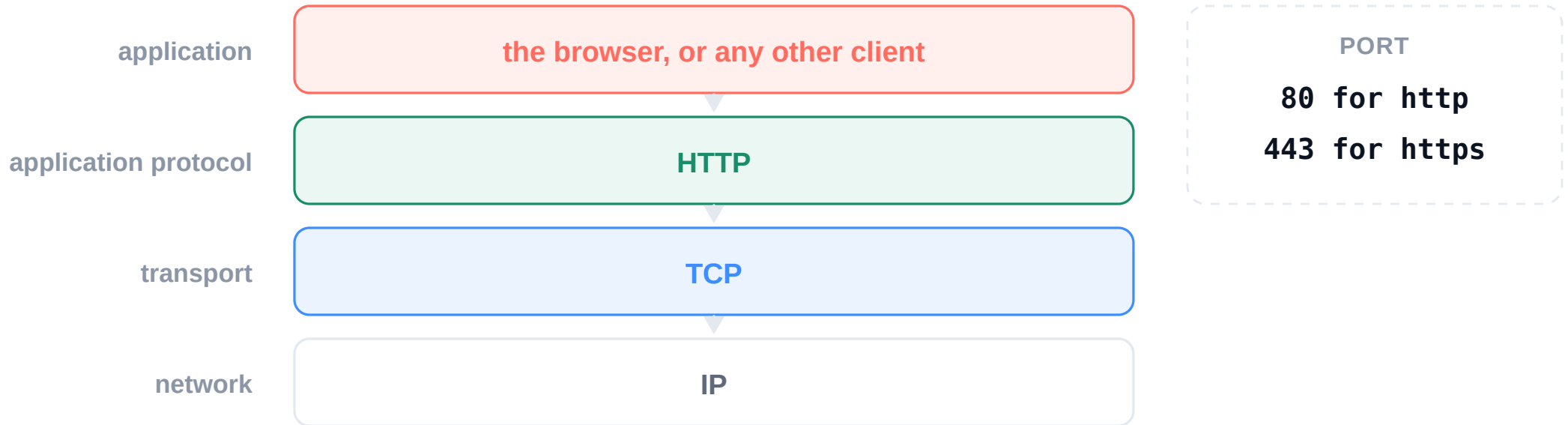
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

HTTP is an application-layer protocol



HTTP defines the messages. The transport below it decides how those messages are carried.

HTTP assumes a reliable, ordered byte stream underneath it.

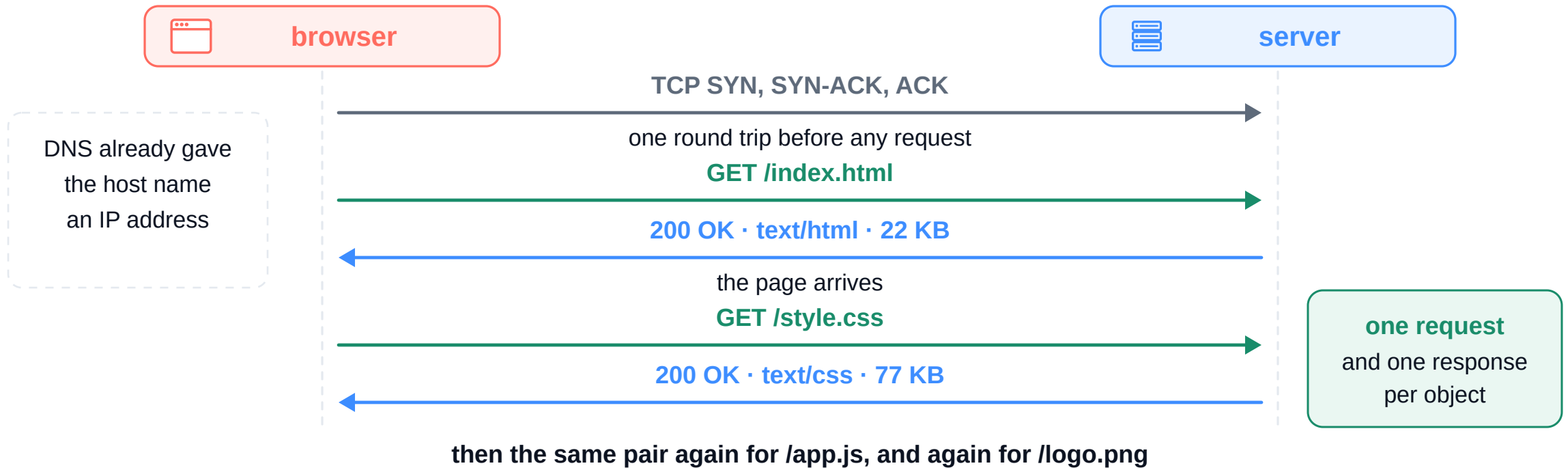
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

What happens when you open one page



Each object costs a request and a response of its own.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Three round trips before the first byte of HTML



3 round trips before the first byte of HTML arrives

On a 40 ms path that is 120 ms in which nothing is transferred.

Both are removed or reduced later in this lecture:

DNS caching · persistent connections

Setup cost is per connection, so connections are worth keeping.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A request is a start line, headers, and a body

THE BYTES ON THE CONNECTION

```
GET /courses/589/index.html HTTP/1.1
```

```
Host: www.cse.buffalo.edu
```

```
User-Agent: Mozilla/5.0
```

```
Accept: text/html
```

```
Accept-Encoding: gzip, br
```

```
(no body for a GET)
```

request line

one line: method, target, version

header fields

name: value, one per line

empty line

CRLF on its own — the headers end here

body

the bytes being sent, if there are any

Every line ends with CR LF. The blank line separates the headers from the body.

Everything before the blank line is metadata; everything after is data.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The first line is a method, a target, and a version

GET

/courses/589/index.html

HTTP/1.1

method

what to do with the object

target

which object, on the host below

version

which rules the sender is using

The target is a path, not a full URL. The host name travels in the Host header instead.

One line states what to do and what to do it to.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

RFC 9110 defines eight methods

METHOD	WHAT IT ASKS THE SERVER TO DO	SAFE	IDEMPOTENT
GET	return the object named by the target	✓	✓
HEAD	return the headers only, no body	✓	✓
POST	process the body as new data for the target	✗	✗
PUT	replace the object with the body	✗	✓
DELETE	remove the object	✗	✓
OPTIONS	report which methods the target supports	✓	✓

Safe: not meant to change anything. Idempotent: sending it twice has the same effect as once. **GET and POST carry almost all traffic.**

The method states the intent; the server decides what it means.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

GET asks the server to send back one file

THE PAGE CONTAINS ``

THE BROWSER SENDS

GET /images/ub-logo.png HTTP/1.1

Host: www.buffalo.edu

THE SERVER SENDS BACK THE FILE

HTTP/1.1 200 OK

Content-Type: image/png

Content-Length: 14203

<14,203 bytes of PNG data>

every image on a page
is fetched with a GET of its own

every link you click
is a GET for the page behind it

the address bar
can only produce a GET

GET does not change anything on the server. Asking twice gives the same answer.

GET means: send me the thing named by this target.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

POST sends data to the server to be processed

A FORM IS FILLED IN AND SUBMITTED

user

student

password

.....

Sign in

THE BROWSER SENDS

POST /login HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 28

user=student&password=secret

the target says what the data is for; the body is the data

THE BODY CAN BE ANYTHING



a form



a file



a JSON document



an image

POST means: here is some data, do something with it.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Host is required, because one address serves many sites



One IP address can host thousands of sites because of this one header.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A response is a status line, headers, and a body

THE BYTES COMING BACK

HTTP/1.1 200 OK

Date: Sat, 05 Sep 2026 14:02:11 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 22147

Cache-Control: max-age=300

<!doctype html><html>...

status line

version, three-digit code, reason text

header fields

what the body is, and how to treat it

empty line

the headers end here

body

the object itself, Content-Length bytes of it

The reason text is for people. Programs read the three digits and ignore the words.

The response says what happened and then sends the object.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The status code is three digits in five classes

1xx **informational**
100 Continue · 103 Early Hints

2xx **the request succeeded**
200 OK · 204 No Content · 206 Partial Content

3xx **look somewhere else, or use your cache**
301 Moved Permanently · 302 Found · 304 Not Modified

4xx **the request was wrong**
400 Bad Request · 403 Forbidden · 404 Not Found · 429 Too Many

5xx **the server failed**
500 Internal Server Error · 502 Bad Gateway · 503 Unavailable

A client that does not know a code must treat it as x00 of the same class.

The first digit is what a client checks first.

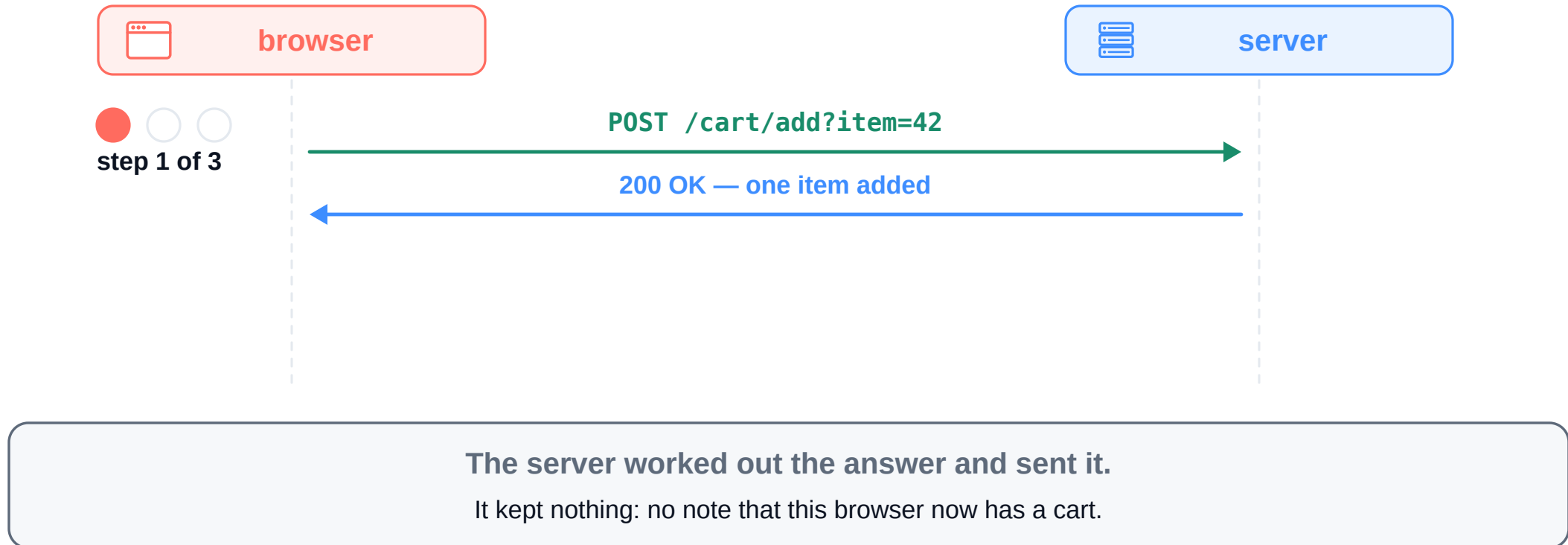
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Nothing in a request refers to the one before it



One request in, one response out, and then it is over.

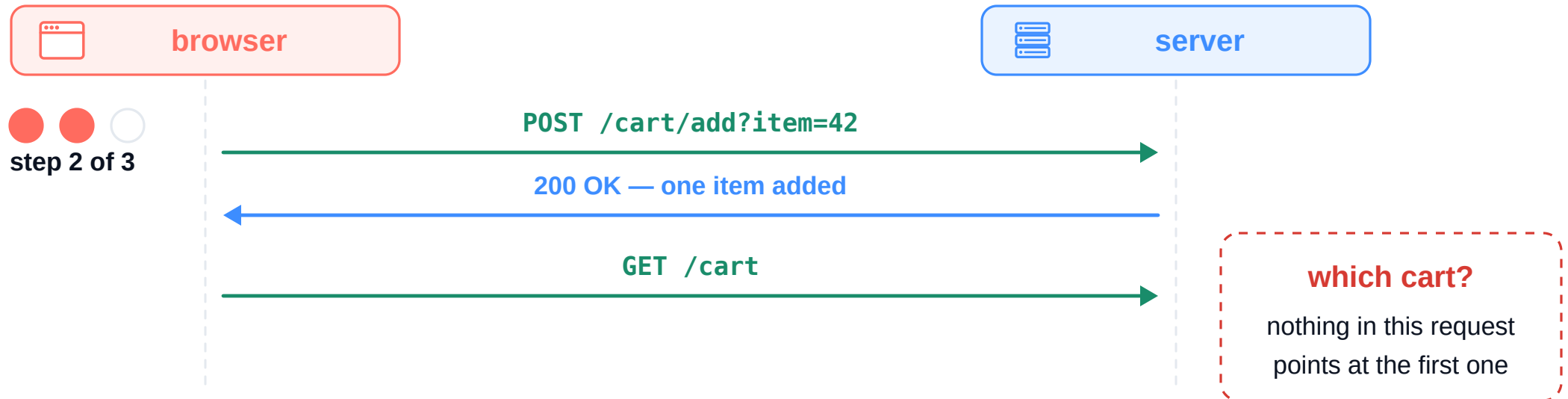
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Nothing in a request refers to the one before it



This request carries a method, a target and headers.

None of them say who is asking, so the server has nothing to match it against.

The server cannot tell that this is the same browser.

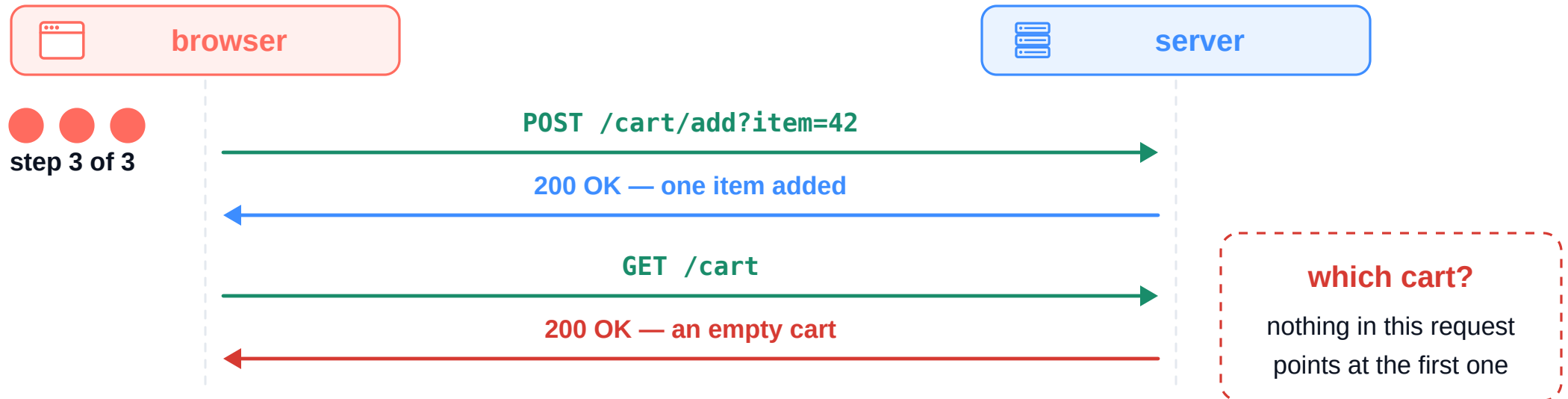
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Nothing in a request refers to the one before it



But a real shopping site does not lose your cart.

Something has to be added to every request to tie it to the one before it. That is Part 4.

HTTP has no memory. Part 4 is about what gets added on top to give it one.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

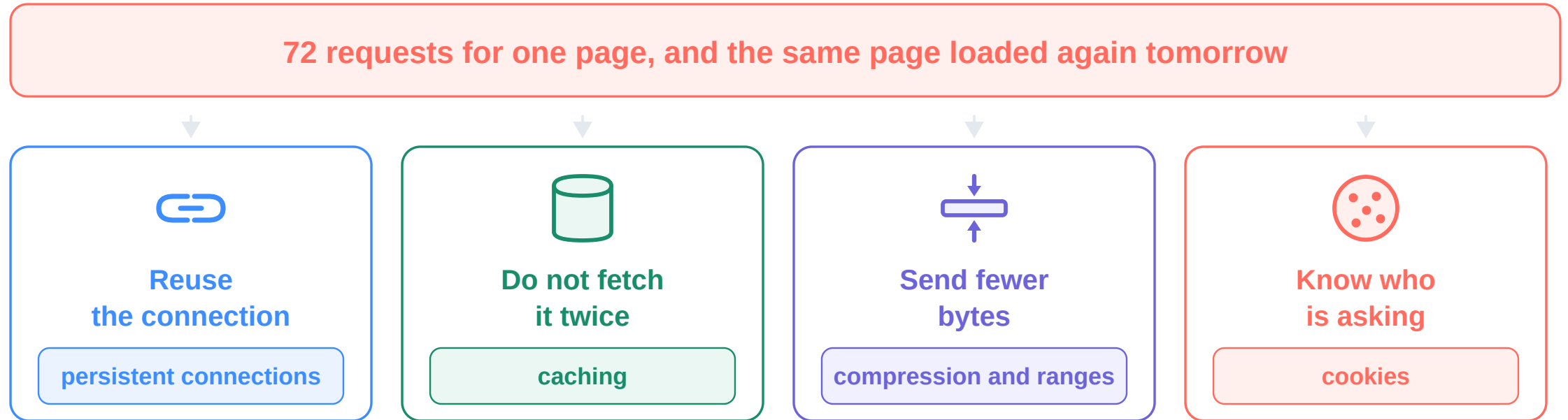
What does HTTP do beyond moving bytes?

04

What HTTP adds

Reusing connections, not fetching an object twice, sending the right form of it, and recognising the same client across requests.

Four mechanisms reduce the cost of a page load



All four are in the protocol itself, not in the browser or the server application.

HTTP is mostly rules about what may be skipped.

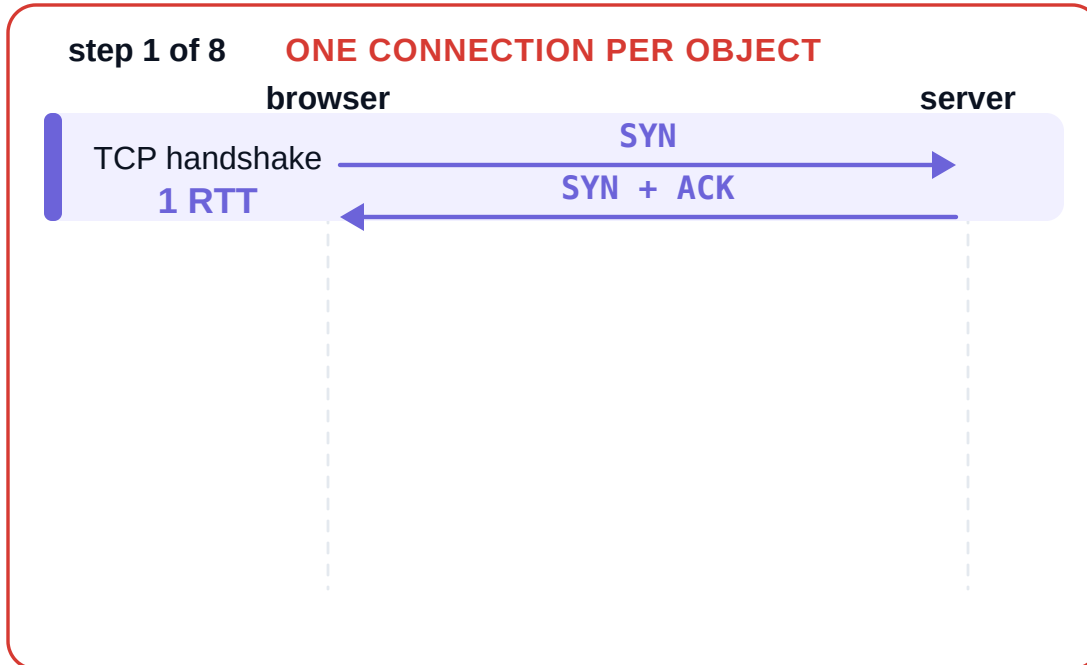
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



the fix comes next

Before any HTTP message can be written, TCP exchanges SYN and SYN-ACK.

That is one round trip in which nothing of the page moves.

A handshake buys nothing but the right to send.

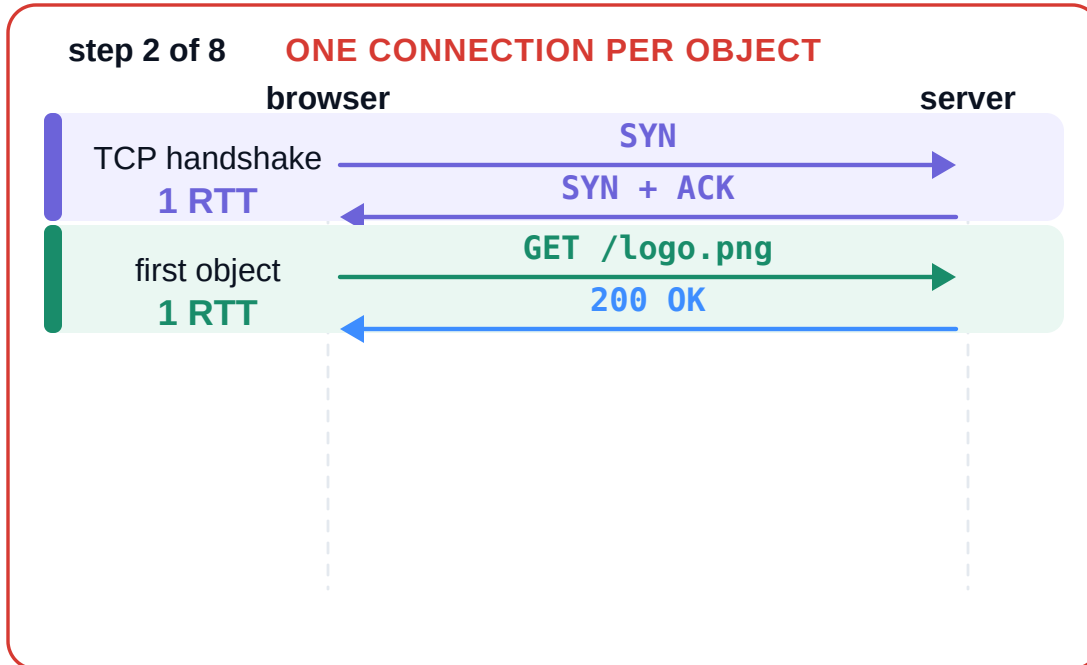
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



the fix comes next

Now the request goes out and the response comes back: one more round trip.

One object has cost two.

One object on its own connection costs two round trips.

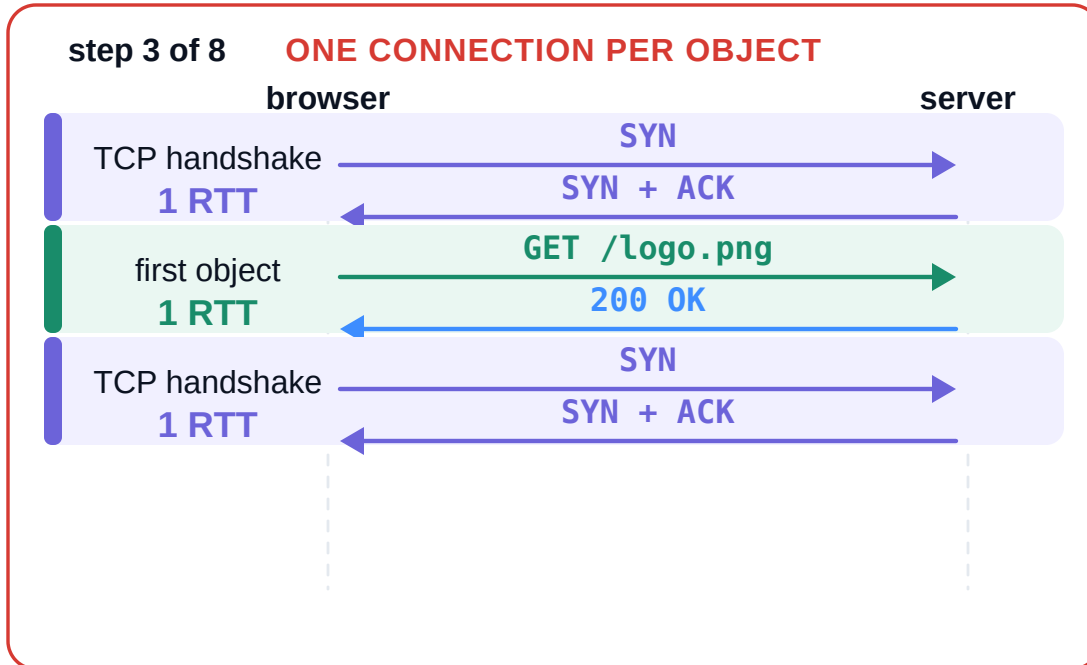
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



the fix comes next

The connection was closed after the response.

The next object starts with another handshake, and pays for it again.

Closing the connection throws the handshake away.

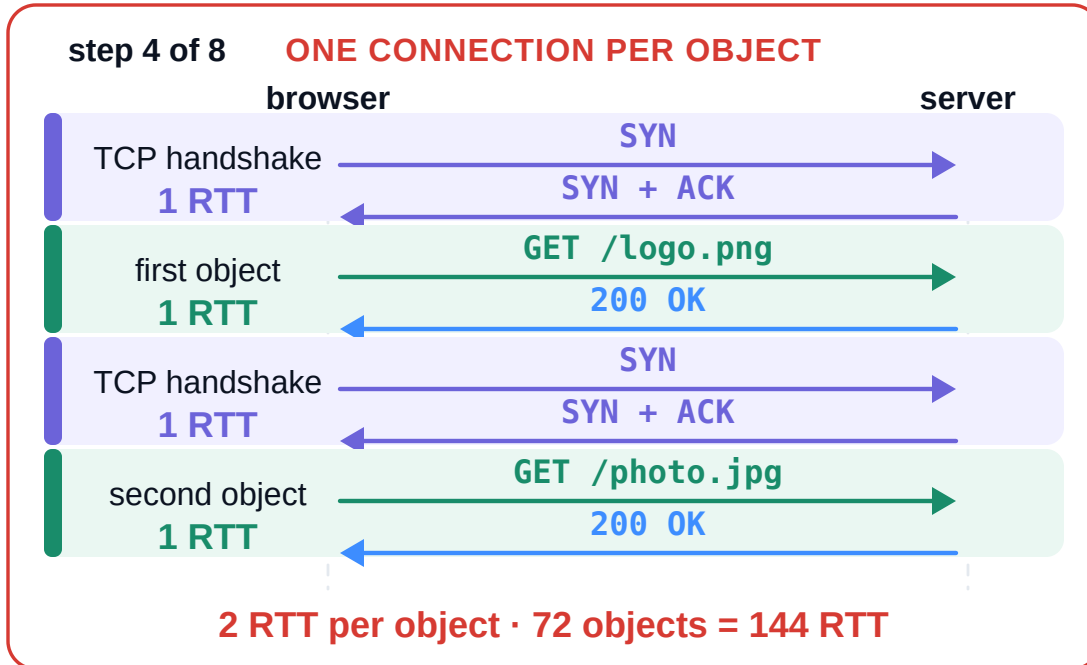
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



the fix comes next

Two round trips for every object.

For the 72 objects of one page that is 144 round trips, before any transfer time.

A connection per object pays two round trips for every file.

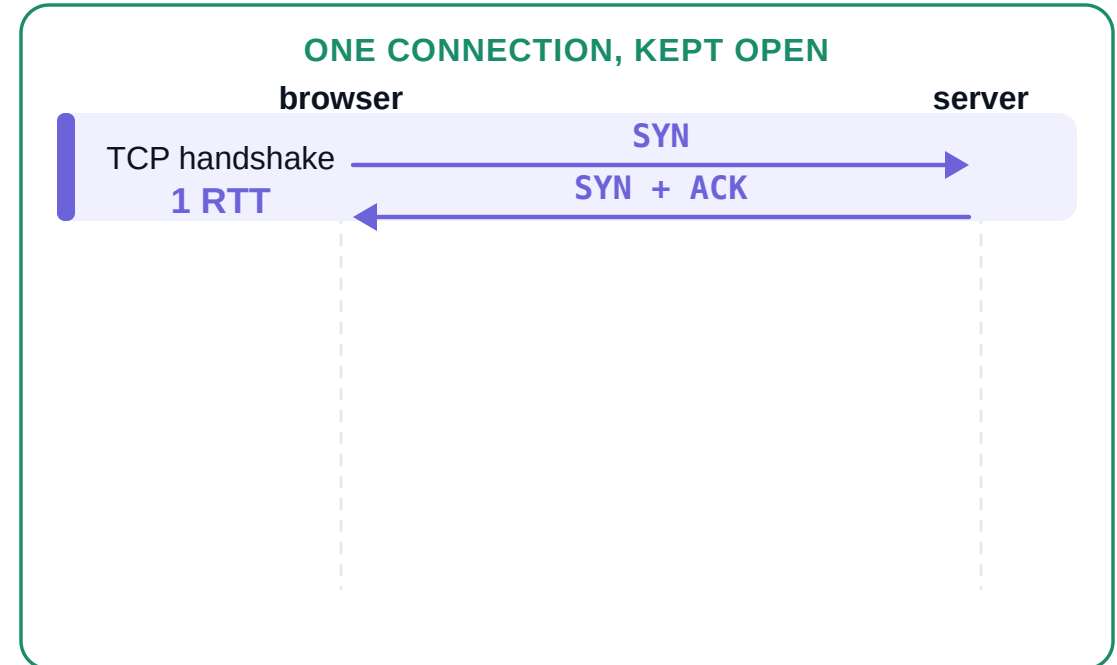
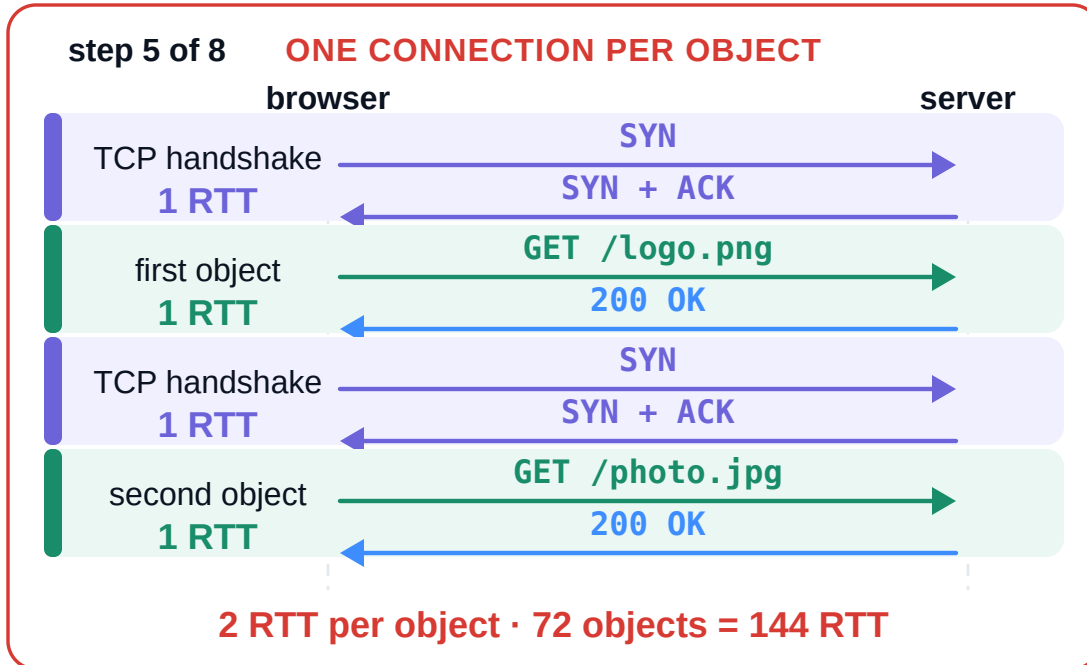
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



Now keep the connection open instead.

The handshake is exactly the same, and it happens once.

The handshake is the same; what changes is how often it happens.

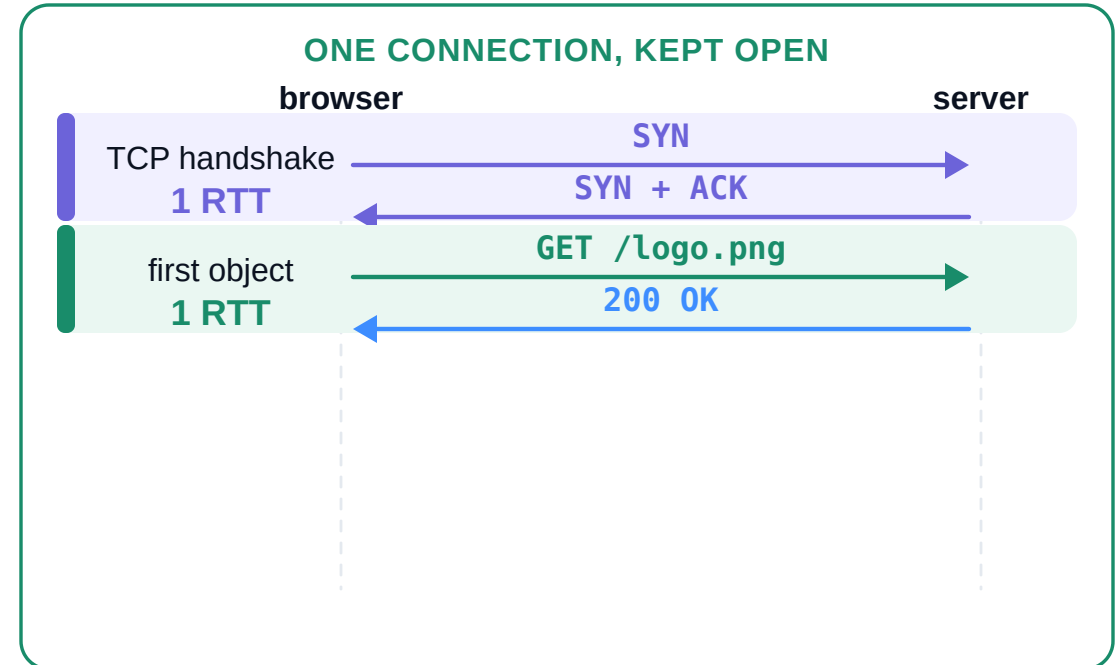
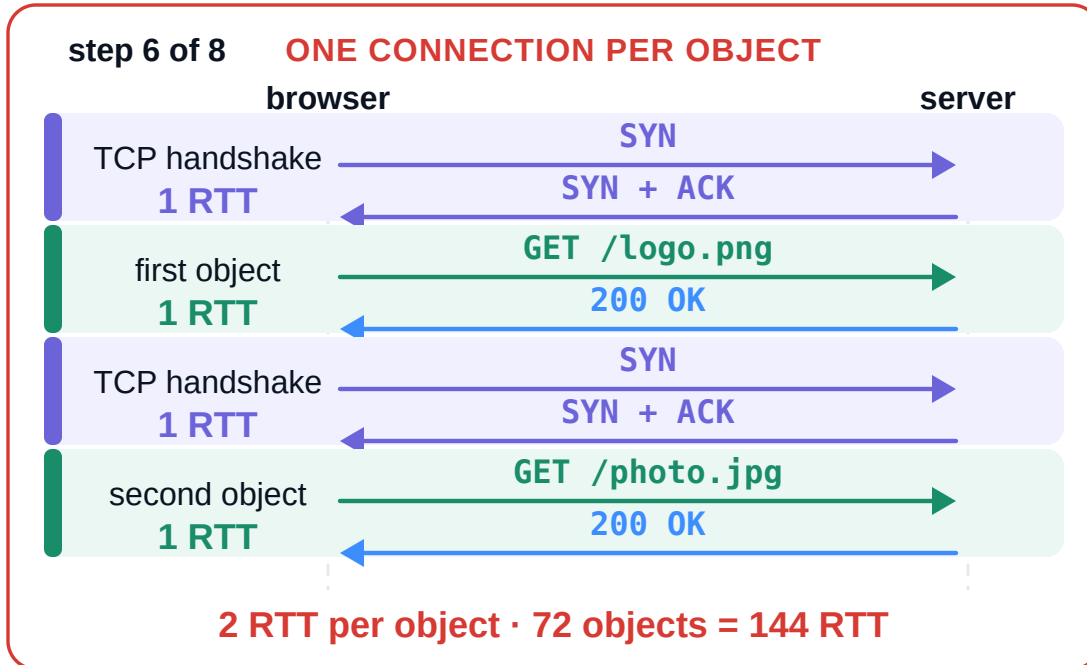
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



The first object costs its one round trip, exactly as it did on the left.
Nothing has been saved yet.

So far this is identical to the left-hand side.

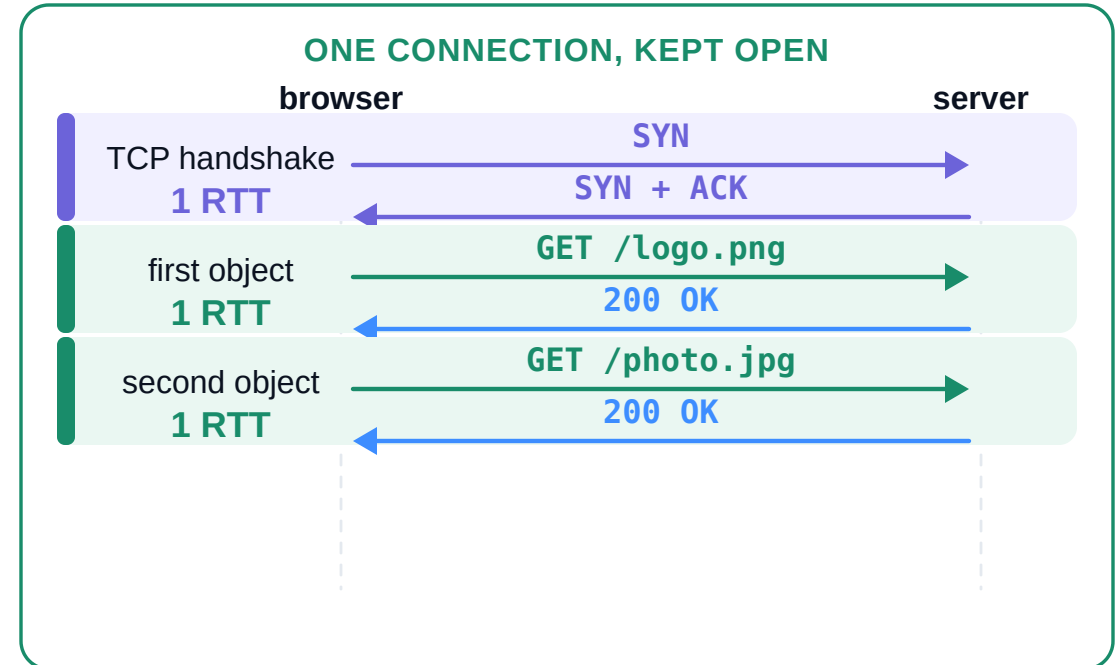
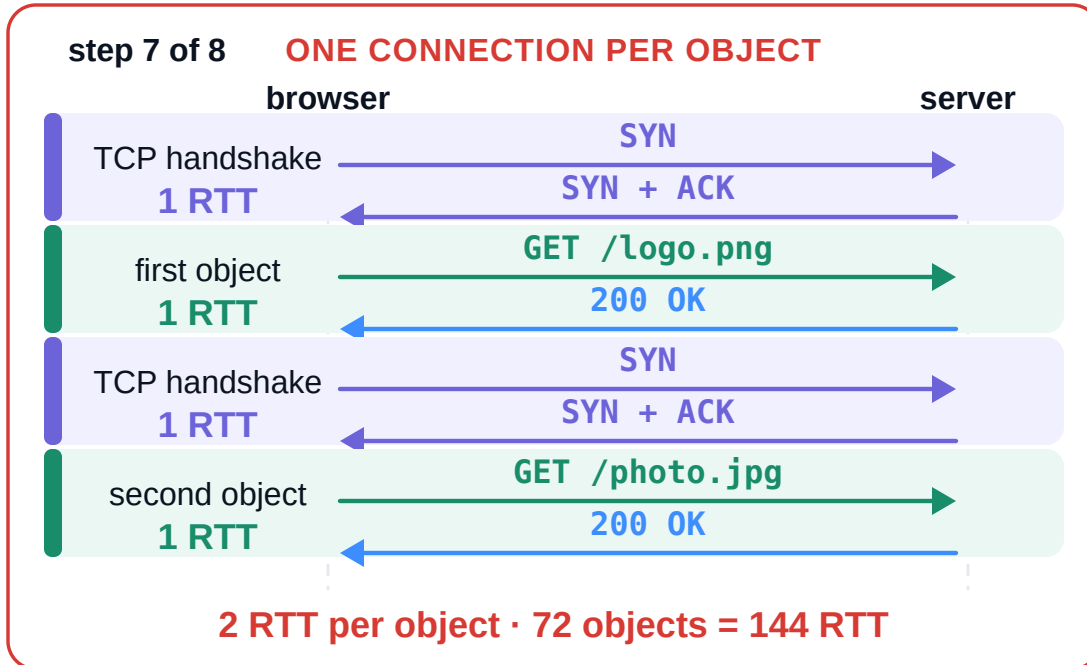
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



The second object goes out on a connection that is already open.
There is no handshake to pay for.

The second object skips the handshake entirely.

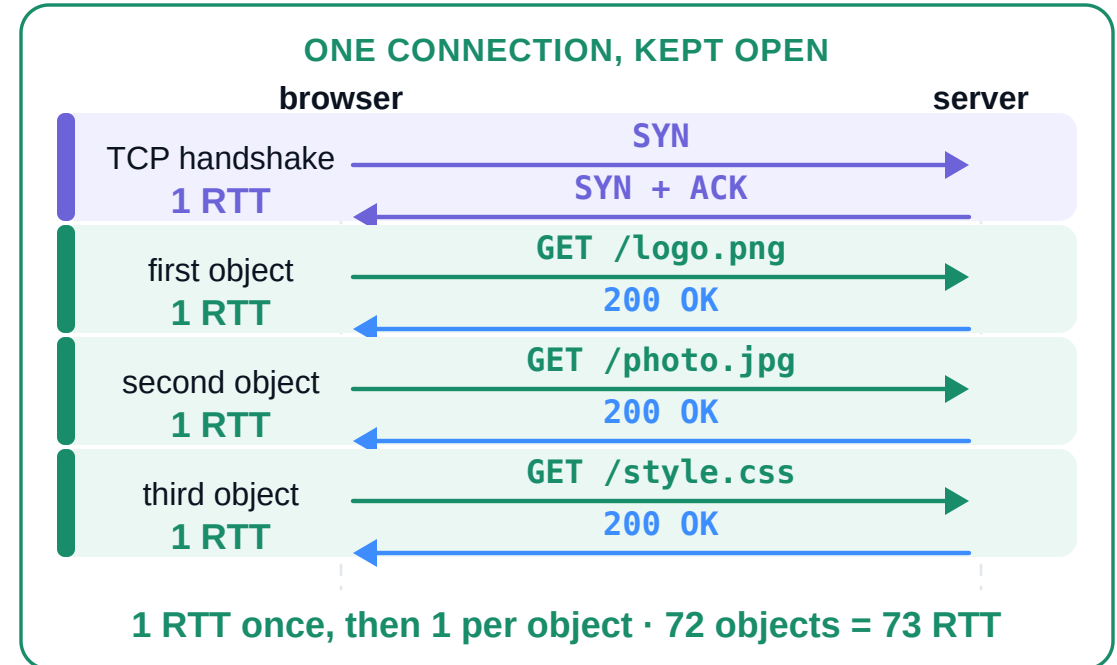
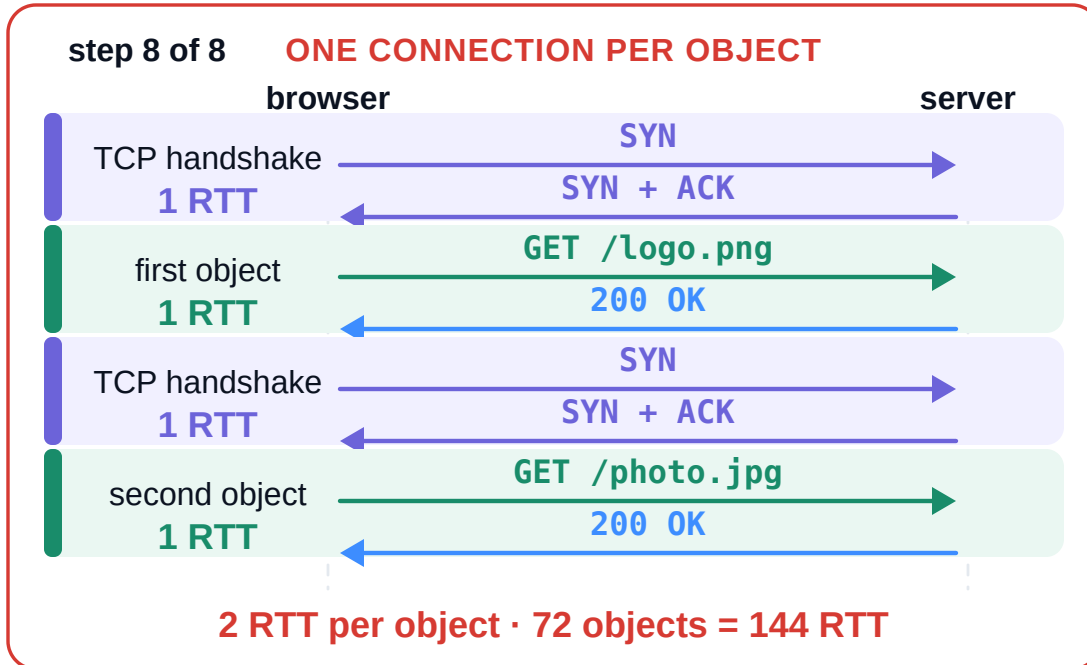
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A handshake costs one round trip, a request costs one more



Every object after the first costs one round trip and nothing else.

For 72 objects that is $1 + 72 = 73$ instead of 144.

Only the first object pays for a handshake.

1 Why the web

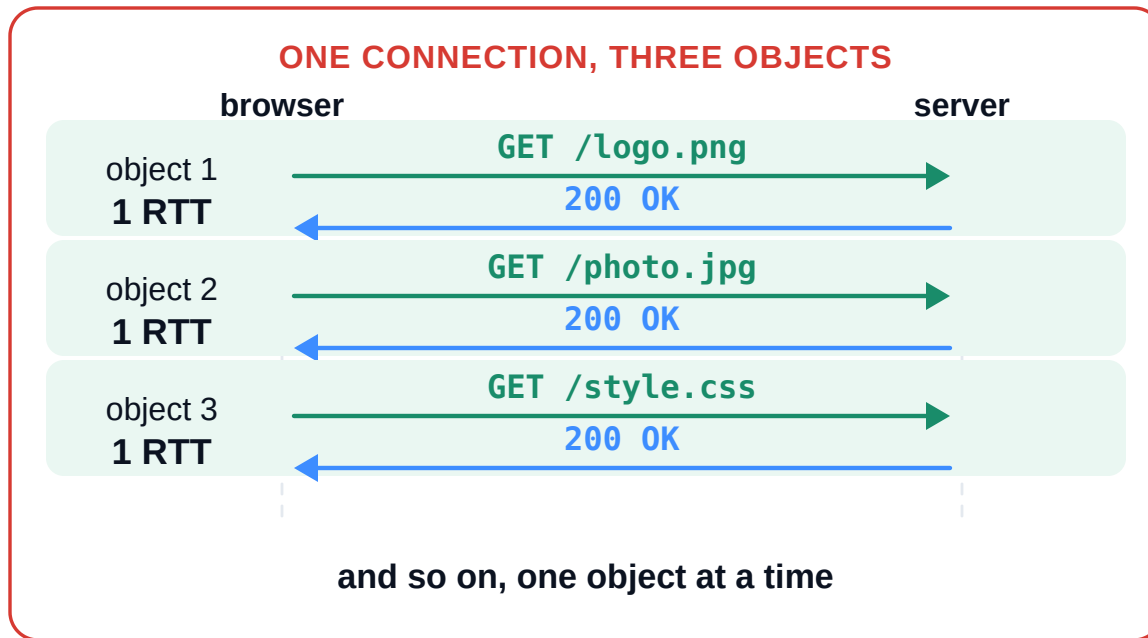
2 How a page works

3 HTTP

4 What HTTP adds

- — One at a time

One connection carries one request at a time



THE RULE IN HTTP/1.1

a request may not be sent until the whole response to the one before it has arrived

SO, FOR ONE PAGE

72 objects × 1 RTT = 72 round trips

on a 40 ms path that is 2.9 seconds
of pure waiting, before a single byte

This is why no browser uses one connection.

One connection is one object at a time.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Browsers open six connections to the same host

WHAT A BROWSER ACTUALLY DOES

connection 1	one object	one object	one object
connection 2	one object	one object	one object
connection 3	one object	one object	one object
connection 4	one object	one object	one object
connection 5	one object	one object	one object
connection 6	one object	one object	one object

one exchange in flight at a time

THE SAME PAGE, 1 CONNECTION

72 objects ÷ 1 = 72 rounds

about 2.9 s of waiting on a 40 ms path

ROUNDS OF WAITING FOR ONE PAGE

1 connection  72



One connection: every object waits for the one before it.

One connection is one object at a time, exactly as on the slide before.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Browsers open six connections to the same host

WHAT A BROWSER ACTUALLY DOES

connection 1	one object	one object	one object
connection 2	one object	one object	one object
connection 3	one object	one object	one object
connection 4	one object	one object	one object
connection 5	one object	one object	one object
connection 6	one object	one object	one object

2 exchanges in flight at once

THE SAME PAGE, 2 CONNECTIONS

72 objects ÷ 2 = 36 rounds

about 1.4 s of waiting on a 40 ms path

ROUNDS OF WAITING FOR ONE PAGE

1 connection  72

2 connections  36



Two connections: two exchanges can be in flight at the same time.

Two connections halve the waiting, and nothing in HTTP changed.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Browsers open six connections to the same host

WHAT A BROWSER ACTUALLY DOES

connection 1	one object	one object	one object
connection 2	one object	one object	one object
connection 3	one object	one object	one object
connection 4	one object	one object	one object
connection 5	one object	one object	one object
connection 6	one object	one object	one object

4 exchanges in flight at once

THE SAME PAGE, 4 CONNECTIONS

72 objects ÷ 4 = 18 rounds

about 0.7 s of waiting on a 40 ms path

ROUNDS OF WAITING FOR ONE PAGE

1 connection		72
2 connections		36
4 connections		18



Four connections: the same page, a quarter of the rounds.

The page is identical. Only the number of connections changed.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Browsers open six connections to the same host

WHAT A BROWSER ACTUALLY DOES

connection 1	one object	one object	one object
connection 2	one object	one object	one object
connection 3	one object	one object	one object
connection 4	one object	one object	one object
connection 5	one object	one object	one object
connection 6	one object	one object	one object

6 exchanges in flight at once

THE SAME PAGE, 6 CONNECTIONS

72 objects ÷ 6 = 12 rounds

about 0.5 s of waiting on a 40 ms path

ROUNDS OF WAITING FOR ONE PAGE

1 connection		72
2 connections		36
4 connections		18
6 connections		12

WHY NOT SIXTY?



each pays its own handshake · they share one link · a server treats more as an attack

HTTP/1.1 gets its parallelism from connections, not from the protocol.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The second visit asks for the same files again

YOUR FIRST VISIT

index.html	22 KB	downloaded
photo.jpg	310 KB	downloaded
logo.png	14 KB	downloaded
font.woff2	28 KB	downloaded
style.css	77 KB	downloaded
app.js	210 KB	downloaded

661 KB downloaded

THE SAME PAGE TOMORROW

index.html	22 KB	changed
photo.jpg	310 KB	identical
logo.png	14 KB	identical
font.woff2	28 KB	identical
style.css	77 KB	identical
app.js	210 KB	identical

22 KB changed · 639 KB identical

The same five files are also sent to every other visitor, and to you again on every page of the site.

90.4% of all responses on the web are marked as safe to store and reuse.

Most of what a page needs was already downloaded once.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A cache stores a response and reuses it

A CACHE

a store of responses: when a request arrives that it has already answered, it returns the stored copy instead of passing the request on.

01

Where are the copies kept?

in the browser, in the network, at a CDN

02

What does a cache actually save?

the transfer, the link, and the trip across the internet

03

Who runs the ones that are not yours?

a campus proxy, or a content delivery network

The rest of this part answers those three questions.

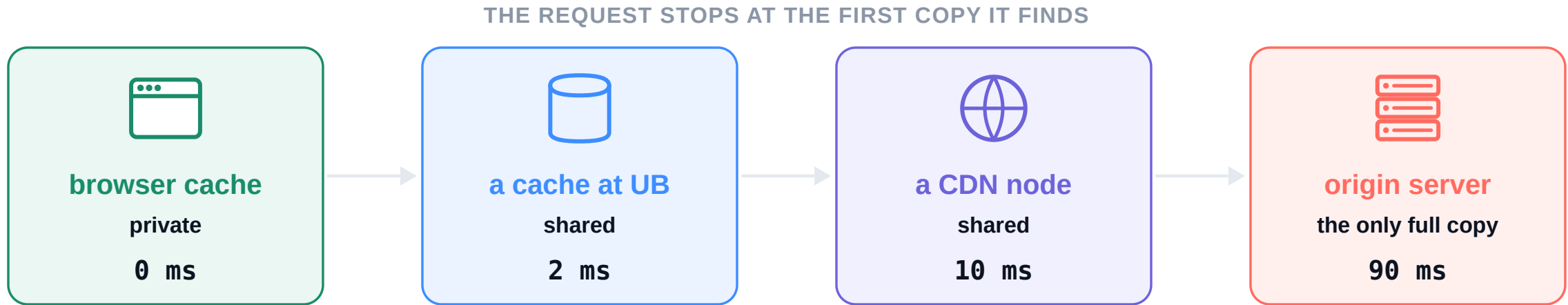
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A request passes up to three caches before the origin



The times are typical round trips from a client in Buffalo.

35% of HTML documents and 71% of third-party objects are served from a CDN

HTTP Archive, 2025 crawl

Caching removes the request, not just the transfer.

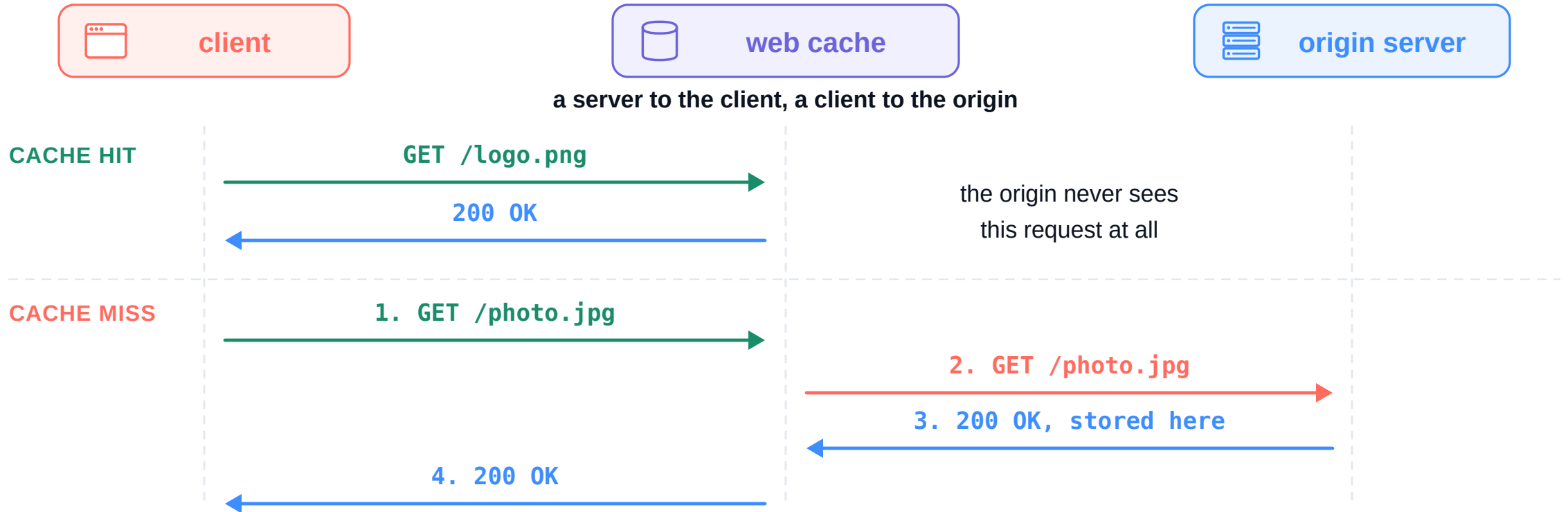
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A web cache answers requests it can, and forwards the rest



On a miss the cache issues its own request and stores what comes back.

1 Why the web

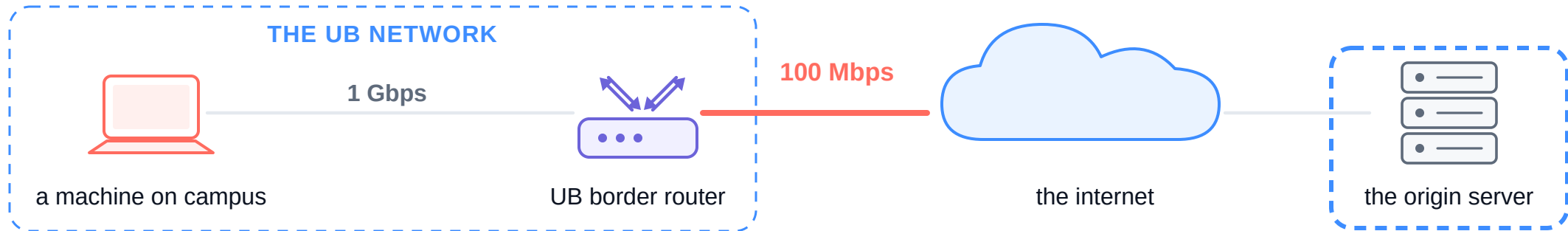
2 How a page works

3 HTTP

4 What HTTP adds

● ——— A worked example

One object, from a campus machine to an origin



the average object	240 kbit	the typical page of Part 2, per object
requests leaving campus	400 per second	across everyone on the network
the link UB pays for	100 Mbps	the only slow link on the path
the internet, router to origin	60 ms	a round trip UB does not control



Everything on this path is measured in bits, so the object is too.

One average object is about 240 kbit.

1 Why the web

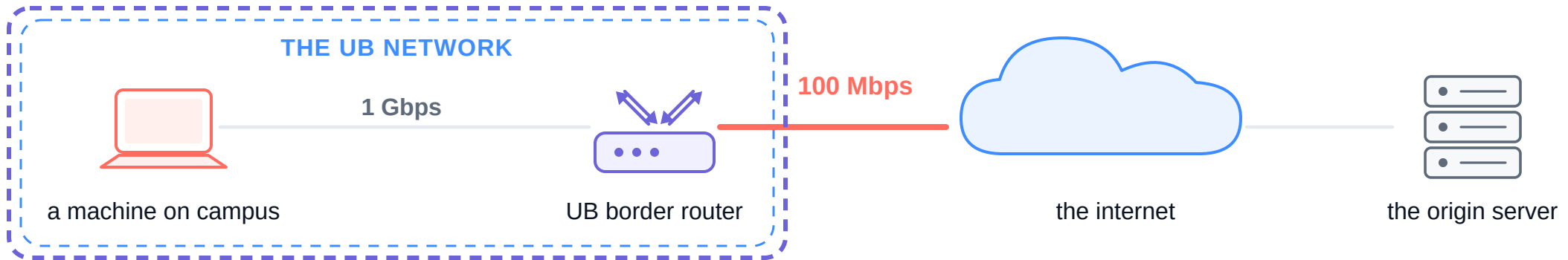
2 How a page works

3 HTTP

4 What HTTP adds

● ——— A worked example

One object, from a campus machine to an origin



the average object

240 kbit

the typical page of Part 2, per object

requests leaving campus

400 per second

across everyone on the network

the link UB pays for

100 Mbps

the only slow link on the path

the internet, router to origin

60 ms

a round trip UB does not control



Everyone on campus together, not one browser.

400 requests a second is the whole campus, not one person.

1 Why the web

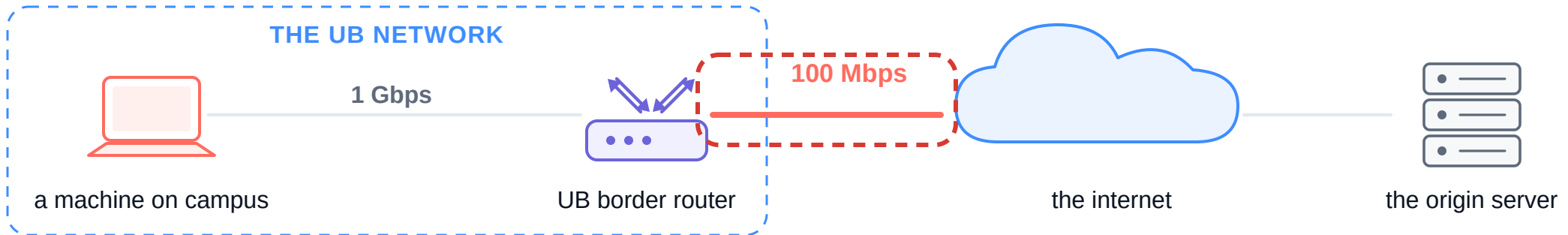
2 How a page works

3 HTTP

4 What HTTP adds

• ——— A worked example

One object, from a campus machine to an origin



the average object

240 kbit

the typical page of Part 2, per object

requests leaving campus

400 per second

across everyone on the network

the link UB pays for

100 Mbps

the only slow link on the path

the internet, router to origin

60 ms

a round trip UB does not control



Every request that leaves campus crosses this one link.

One link on the path is slower than everything else.

1 Why the web

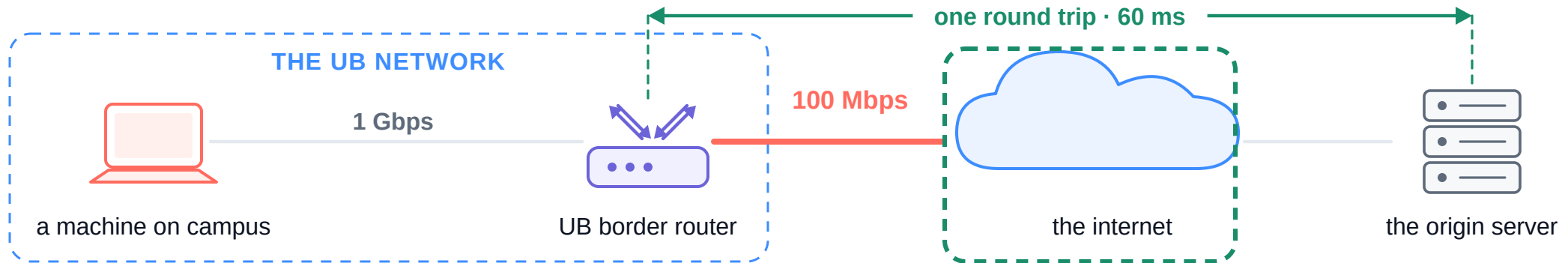
2 How a page works

3 HTTP

4 What HTTP adds

• — A worked example

One object, from a campus machine to an origin



the average object

240 kbit

the typical page of Part 2, per object

requests leaving campus

400 per second

across everyone on the network

the link UB pays for

100 Mbps

the only slow link on the path

the internet, router to origin

60 ms

a round trip UB does not control



How long does one object take, from the request to its last byte?

Four numbers, and one question to answer with them.

1 Why the web

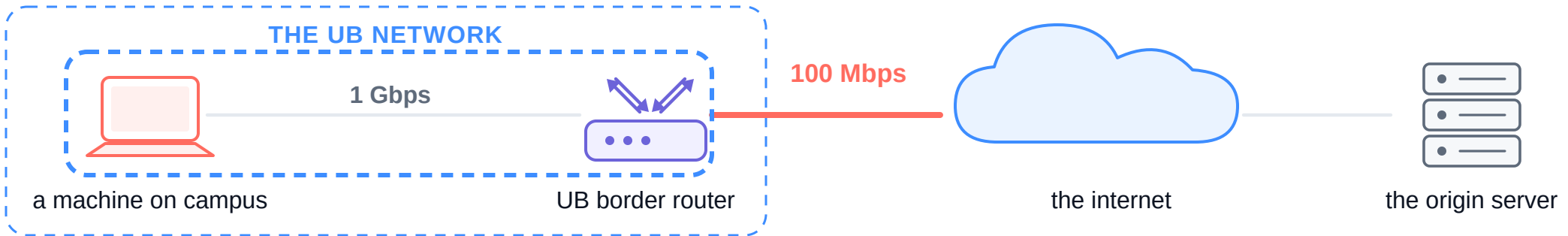
2 How a page works

3 HTTP

4 What HTTP adds

● ——— Where the time goes

The delay has three parts, and one is unknown



THE CAMPUS LAN

0.24 ms

$240 \text{ kbit} \div 1 \text{ Gbps}$

THE 100 Mbps LINK

THE INTERNET



240 kbit over the 1 Gbps campus network.

The campus network is fast enough to ignore.

1 Why the web

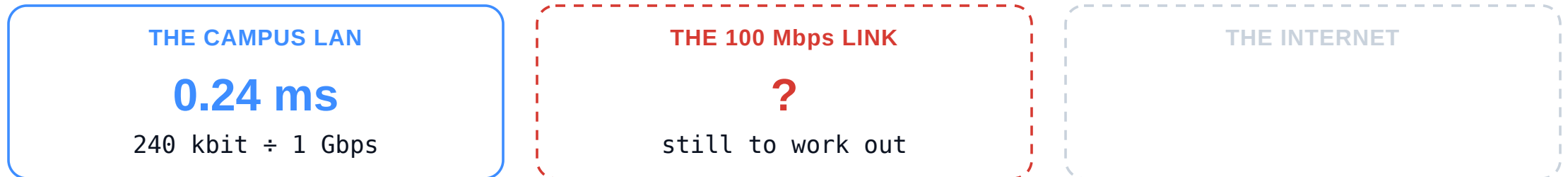
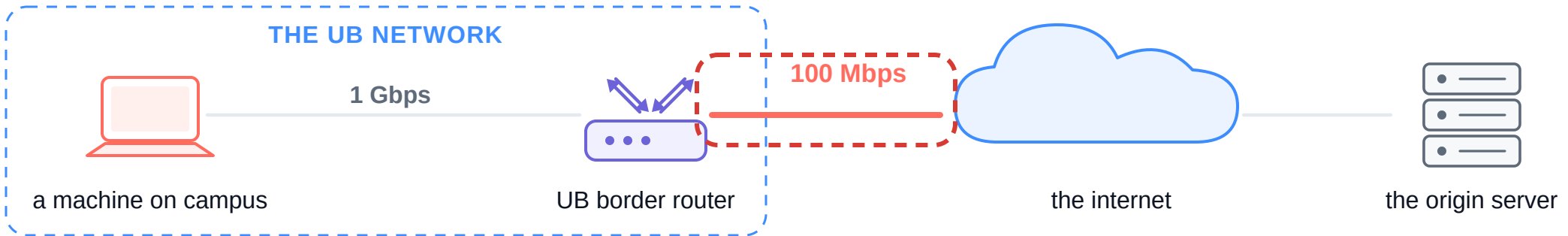
2 How a page works

3 HTTP

4 What HTTP adds

• ——— Where the time goes

The delay has three parts, and one is unknown



The only part of the path we do not know yet.

The middle one is the only unknown.

1 Why the web

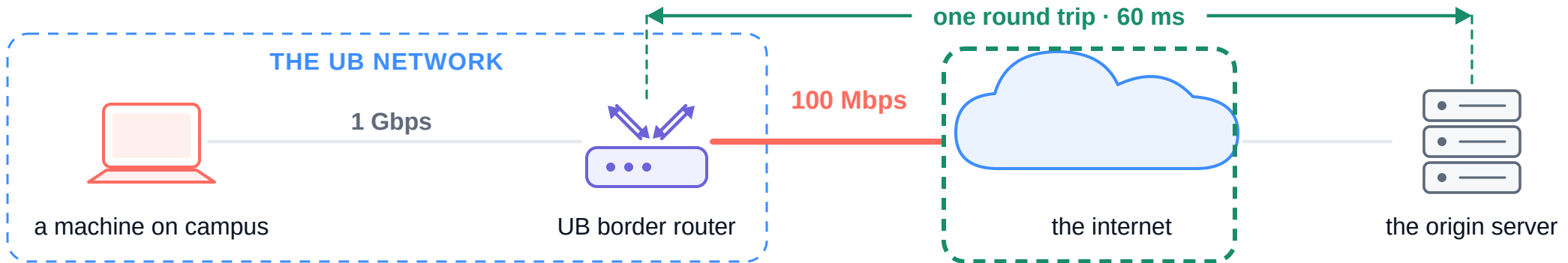
2 How a page works

3 HTTP

4 What HTTP adds

● ——— Where the time goes

The delay has three parts, and one is unknown



THE CAMPUS LAN

0.24 ms

$240 \text{ kbit} \div 1 \text{ Gbps}$

THE 100 Mbps LINK

?

still to work out

THE INTERNET

60 ms

round trip, fixed



A round trip out to the origin and back, which UB does not control.

Work out the middle one and the question is answered.

1 Why the web

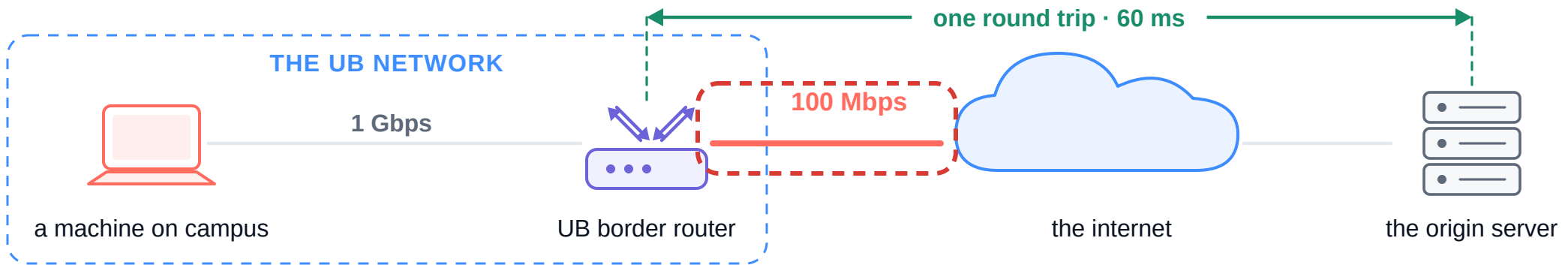
2 How a page works

3 HTTP

4 What HTTP adds

- — The link, worked out

Working out the time on the 100 Mbps link



01 one object on the link $240 \text{ kbit} \div 100 \text{ Mbps}$ **2.4 ms**

02 offered by the whole campus $400 \text{ /s} \times 240 \text{ kbit}$ **96 Mbps**

03 so the link keeps up $96 \text{ Mbps} < 100 \text{ Mbps}$ **no waiting**



One object takes 2.4 ms on this link, and the whole campus offers it 96 Mbps.

Two numbers about the link: one object, and everything the campus sends.

1 Why the web

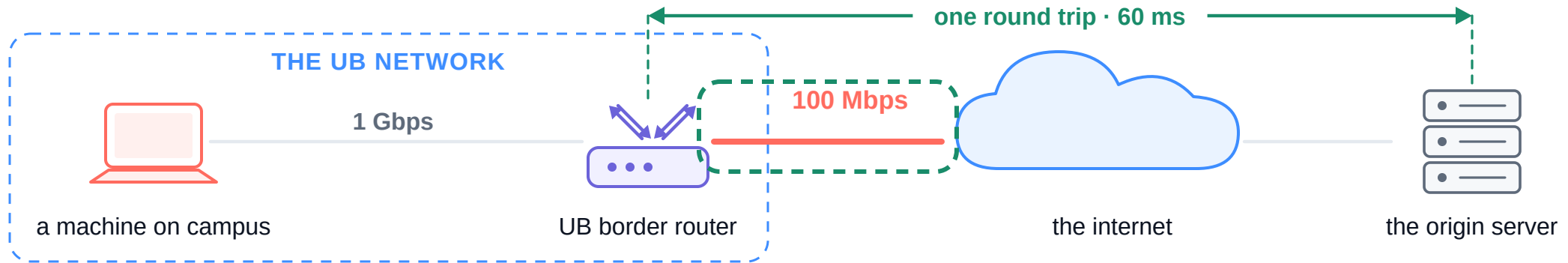
2 How a page works

3 HTTP

4 What HTTP adds

- — The link, worked out

Working out the time on the 100 Mbps link



01 one object on the link $240 \text{ kbit} \div 100 \text{ Mbps}$ **2.4 ms**

02 offered by the whole campus $400 \text{ /s} \times 240 \text{ kbit}$ **96 Mbps**

03 so the link keeps up $96 \text{ Mbps} < 100 \text{ Mbps}$ **no waiting**



Under the capacity, so we assume an object goes out with nothing in front of it.

Past 100 Mbps the queue never drains and this sum stops working.

1 Why the web

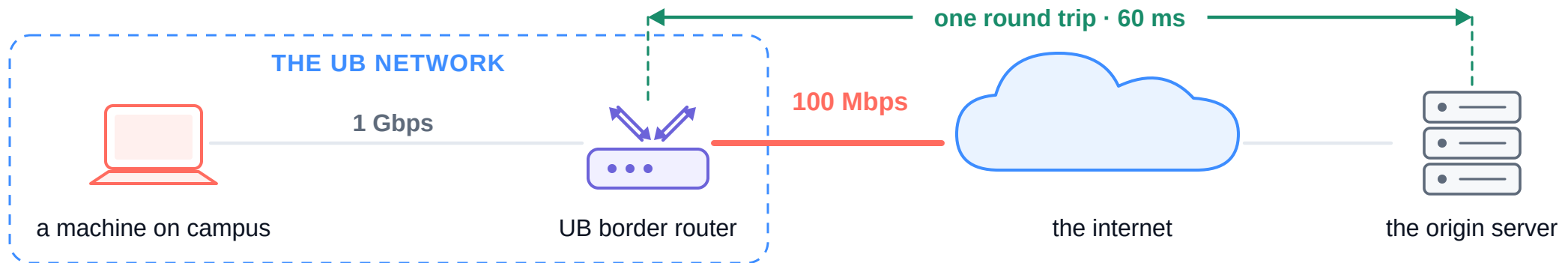
2 How a page works

3 HTTP

4 What HTTP adds

- — The link, worked out

Working out the time on the 100 Mbps link



01	one object on the link	$240 \text{ kbit} \div 100 \text{ Mbps}$	2.4 ms
----	------------------------	--	--------

02	offered by the whole campus	$400 /s \times 240 \text{ kbit}$	96 Mbps
----	-----------------------------	----------------------------------	---------

03	so the link keeps up	$96 \text{ Mbps} < 100 \text{ Mbps}$	no waiting
----	----------------------	--------------------------------------	------------



$$0.24 \text{ ms} + 2.4 \text{ ms} + 60 \text{ ms} = 62.6 \text{ ms}$$

the campus LAN, the link, and the internet, for one object

One object takes about 63 ms, and 60 of them are the internet.

1 Why the web

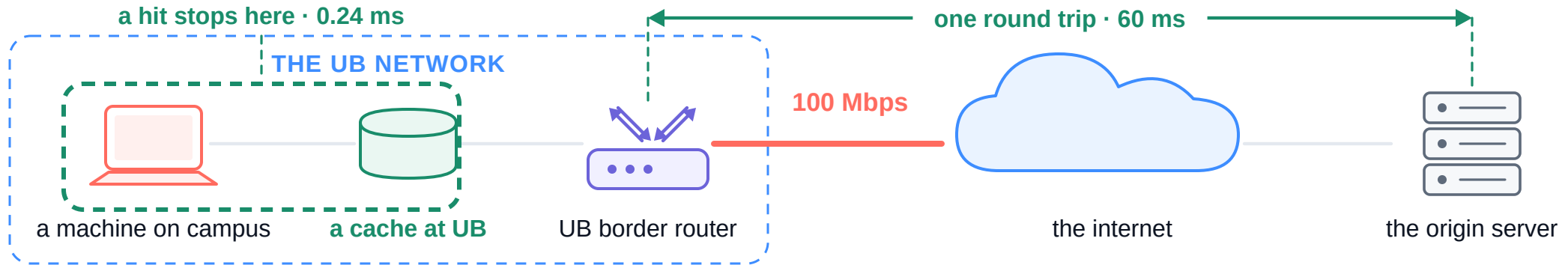
2 How a page works

3 HTTP

4 What HTTP adds

• — A cache at UB

A hit is answered on campus and costs 0.24 ms



still leaving campus

60% of 400 /s × 240 kbit

58 Mbps

the link still keeps up

58 Mbps < 100 Mbps

no waiting

a hit, a miss

0.24 ms and 62.6 ms

as before

$0.6 \times 62.6 \text{ ms} + 0.4 \times 0.24 \text{ ms} = 37.7 \text{ ms on average}$

four requests in ten never leave the campus at all

A 40% hit rate takes the average from 62.6 ms to 37.7 ms.

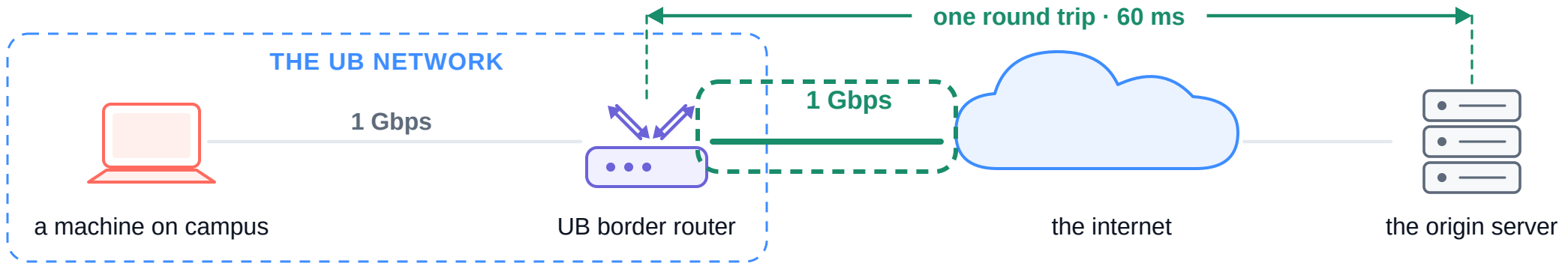
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Ten times the link speed removes 2.2 ms



one object on the link	$240 \text{ kbit} \div 1 \text{ Gbps}$	0.24 ms
offered by the campus	$96 \text{ Mbps} < 1000 \text{ Mbps}$	no waiting
what the upgrade removed	$2.4 \text{ ms} - 0.24 \text{ ms}$	2.2 ms
$0.24 \text{ ms} + 0.24 \text{ ms} + 60 \text{ ms} = 60.5 \text{ ms}$ ten times the link speed, and every request still crosses the internet		

The link was never where the time went, so making it faster changes little.

1 Why the web

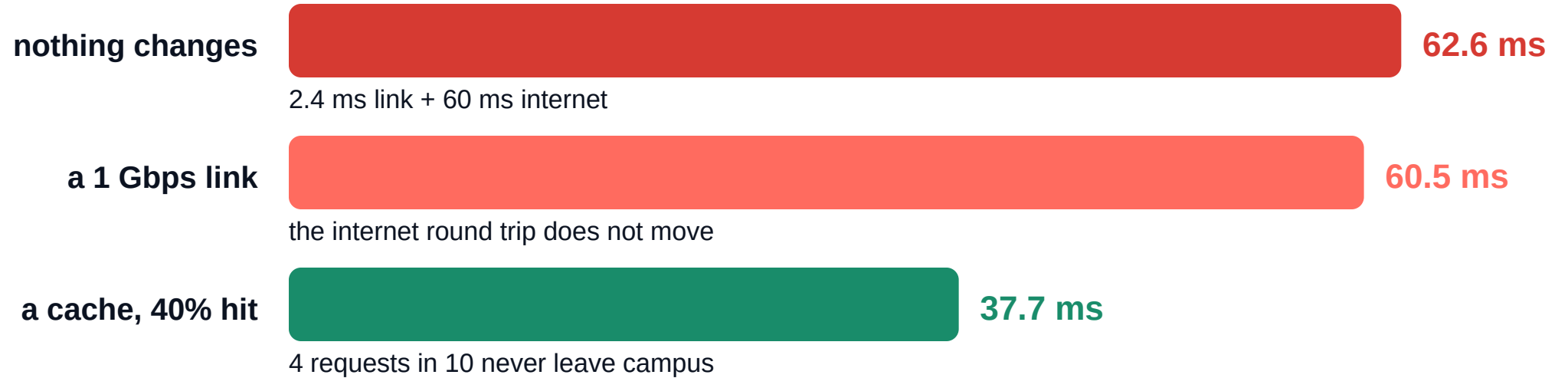
2 How a page works

3 HTTP

4 What HTTP adds

A cache beats a link ten times faster

AVERAGE TIME FOR ONE OBJECT



A faster link removes 2.2 ms. A cache removes the whole 60 ms, for 40% of the requests.

The bottleneck was never the link UB pays for. It is the round trip out to the origin, and the only way to avoid that is to not make the trip.

Caching is the only one of the two that removes the request.

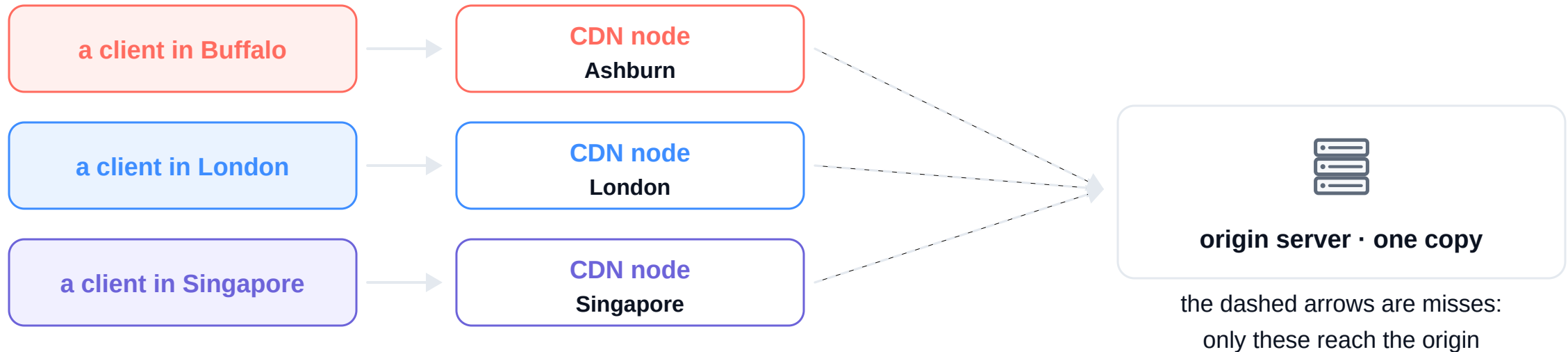
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A CDN is the same caching server, in many places



Age: 312

the only header that says a response came from a cache

DNS is what sends each client
to a nearby node.

Caching is what makes one origin serve the whole world.

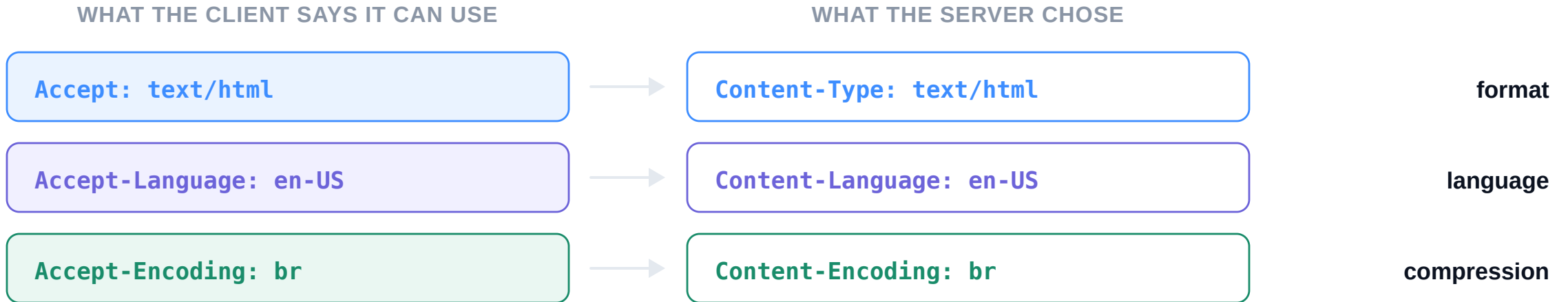
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

One URL can return different forms of the same object



Vary: Accept-Encoding, Accept-Language

the response depends on these request headers, so a cache must key on them too
without Vary a shared cache would hand the English page to everyone

Vary tells a cache which request headers are part of the key.

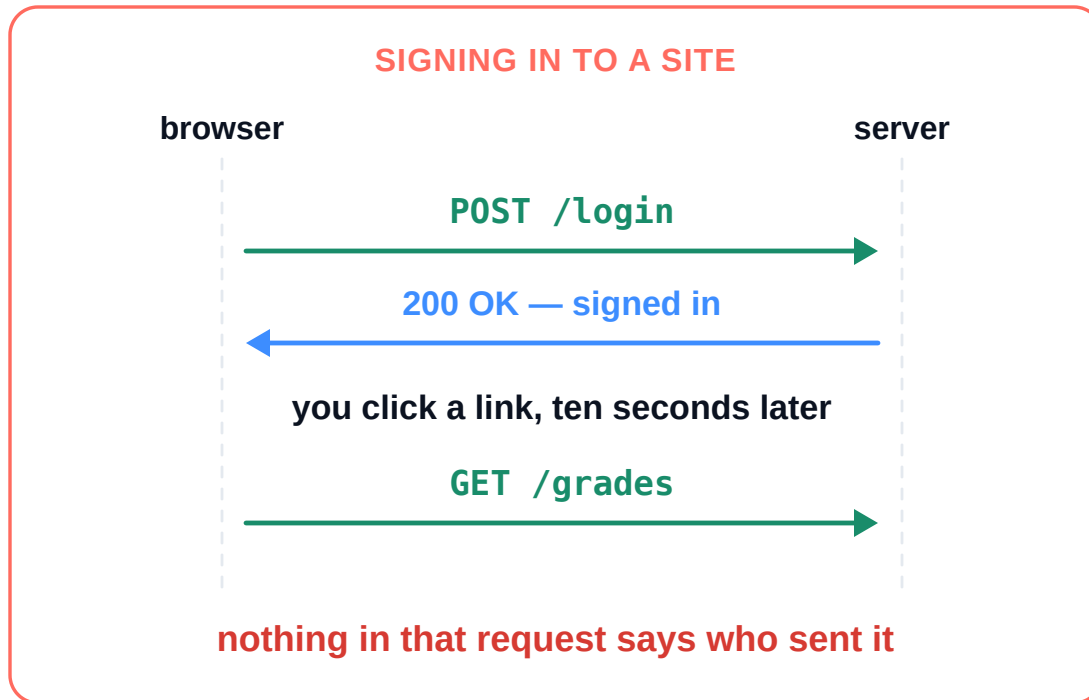
1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

You sign in once, and every click after that still counts



- you stay signed in**
the password was typed on a different request
- your cart keeps its items**
they were added on a different request
- the site remembers dark mode**
you chose it on a different day
- all three need the same browser recognised**

Every site you use depends on this, and HTTP does not provide it.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

The application needs memory the protocol does not have

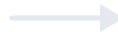
WHAT THE SERVER NEEDS

this is the same person who put
two items in the cart four requests ago
and who logged in before that

WHAT HTTP PROVIDES

nothing

each request is processed on its own



the client carries the state

the server sends a value once,
and the client returns it on
every later request



Defined in RFC 6265.

Every request still carries everything needed to answer it — now including who sent it.
So any of the site's servers can take the next one; none of them has to remember you.

A cookie is state stored on the client and sent back every time.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

One header sets the value, another returns it



The value means nothing to the browser. It stores what the server sent and returns it unchanged.

The cookie identifies the client; the data stays on the server.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

A typical page carries nine cookies



WHO SET THEM

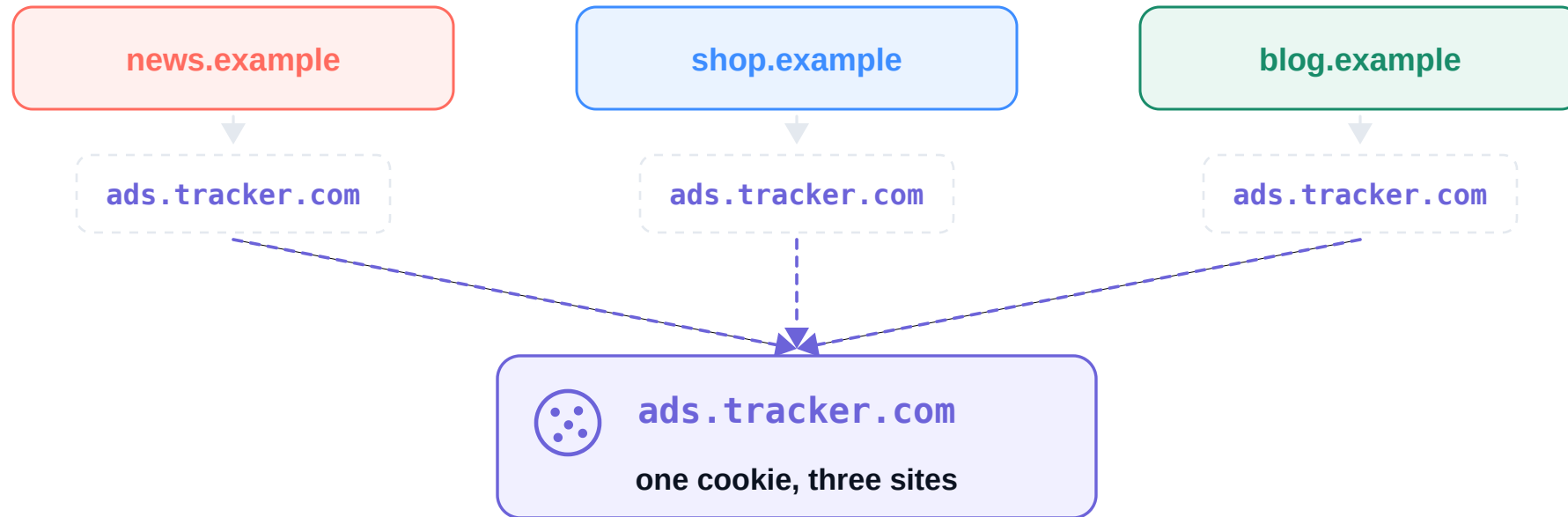


The 4,096-byte limit is what stops a cookie from being used as storage.

Most cookies on a page were not set by the site being visited.

- 1 Why the web
- 2 How a page works
- 3 HTTP
- 4 What HTTP adds

A cookie set by an embedded object follows the browser



Every page that embeds the same third-party object sends that third party the same cookie, so it learns which sites this browser visited.

The cookie belongs to the host it was set by, not to the page.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds

Four questions to check yourself against

01 What is the unit HTTP transfers, and how is it named?

02 Which parts of a message are metadata, and which are data?

03 Which mechanism removes a request, and which only shortens it?

04 Where may a copy of a response legitimately be kept?

The same four questions apply to any application protocol.

Everything in Part 4 exists to avoid sending something twice.

1 Why the web

2 How a page works

3 HTTP

4 What HTTP adds